

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
Национальный исследовательский ядерный университет
«МИФИ»



Институт интеллектуальных
кибернетических систем
КАФЕДРА КИБЕРНЕТИКИ

Пояснительная записка

к учебно-исследовательской работе студента на тему:
Разработка прототипа системы синхронизации файлов для системы поддержки
учебного процесса

Направление	_____	09.03.04
подготовки	_____	ПРОГРАММНАЯ
		ИНЖЕНЕРИЯ
Студент	_____	Гаина Георгий

Группа	_____	Б23-514

Научный	_____	
руководитель	_____	Рословцев В.В.
Научный	_____	
консультант	_____	
Оценка	_____	
руководителя	_____	
Оценка комиссии	_____	

Москва, 2026

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
Национальный исследовательский ядерный университет
«МИФИ»



Институт интеллектуальных
кибернетических систем
КАФЕДРА КИБЕРНЕТИКИ

Задание на УИР

Студенту гр. Б23-514
(группа)

Гаина Георгий Евгеньевич
(фио)

ТЕМА УИР

Разработка прототипа системы синхронизации файлов для системы поддержки учебного процесса

ЗАДАНИЕ

№ п/п	Содержание работы	Форма отчетности	Срок исполнения	Отметка о выполнении и Дата, подпись рук.
1.	Аналитическая часть			
1.1.	Анализ возможностей платформы .NET для манипуляции файловой системой с точки зрения создания виртуальных папок.	Разделы ПЗ	01.04.2026	
1.2.	Анализ возможностей .NET для разработки систем распределённого хранения на основе асинхронного взаимодействия.	Разделы ПЗ	01.04.2026	
1.3.	Анализ особенностей децентрализованных архитектур и подходов к хранилищам неструктурированных данных.	Разделы ПЗ	05.04.2026	
1.4.	Сравнительный анализ подходов к версионированию объектов (Git, SVN и аналоги).	Разделы ПЗ	10.04.2026	
1.5.	Выводы по аналитической части.	Разделы ПЗ	12.04.2026	
2.	Теоретическая часть			
2.1.	Разработка моделей метаданных для хранения и версионирования неструктурированных данных.	Модели, разделы ПЗ	20.04.2026	
2.2.	Разработка моделей объектов данных и метаданных для версионирования структурированных объектов.	Модели, таблицы, разделы ПЗ	25.04.2026	

	информации об объектах на малом экране.			
2. 3.	Разработка алгоритмов синхронизации версий в распределённой среде и разрешения конфликтов (merge, conflict resolution).	Алгоритмы, схемы, разделы ПЗ	30.04.2026	
3.	Инженерная часть			
3. 1.	Проектирование архитектуры децентрализованной системы хранения неструктурированных данных.	Архитектура, схемы, разделы ПЗ	05.05.2026	
3. 2.	Проектирование фрагмента схемы базы данных для описания версий на разных узлах	ER-диаграммы, разделы ПЗ	10.05.2026	
3. 3.	Проектирование программных интерфейсов (API) взаимодействия узлов синхронизации.	Программный интерфейс, разделы ПЗ	12.05.2026	
4.	Технологическая и практическая часть			
4. 1.	Разработка программного прототипа системы распределённого хранения и синхронизации.	Исходные тексты, исполняемые файлы	16.05.2026	
4. 2.	Механизмы загрузки данных и наполнение тестовыми файлами.	Тестовые данные, разделы ПЗ	20.05.2026	
4. 3.	Отладка, функциональное тестирование и оценка производительности.	Рабочие материалы, разделы ПЗ	20.05.2026	
5.	<i>Оформление пояснительной записки (ПЗ) и иллюстративного материала для доклада.</i>	Текст ПЗ, презентация	22.05.2026	

ЛИТЕРАТУРА

1.	Fowler M. Patterns of Enterprise Application Architecture. — Boston: Addison-Wesley, 2002. — 560 p
2.	Nielsen J. Usability Engineering. — Morgan Kaufmann, 1994.
3.	Gamma E., Helm R., Johnson R., Vlissides J. Design Patterns: Elements of Reusable Object-Oriented Software. — Boston: Addison-Wesley, 1994. — 395 p
4.	Larman C. Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design. — Prentice Hall, 2004. — 720 p.
5.	Evans E. Domain-Driven Design: Tackling Complexity in the Heart of Software. — Boston: Addison-Wesley, 2003. — 560 p.
6.	Chacon S., Straub B. Pro Git. — 2nd ed. — Apress, 2014. — 458 p.
7.	Troelsen A., Japikse P. Pro C# 10 with .NET 6: Foundational Principles and Practices in Programming. — Apress, 2022. — 1400 p

Дата выдачи задания:

« 15 » февраля 2026г.

Руководитель _____

Студент _____

Рословцев В.В.

Гаина Г.Е.

Реферат

Пояснительная записка содержит: 53 страницы, 12 таблиц, 9 рисунков, 4 формулы, 21 использованный источник, 1 приложение.

СИСТЕМА СИНХРОНИЗАЦИИ ФАЙЛОВ, РАСПРЕДЕЛЁННОЕ ХРАНЕНИЕ, ВЕРСИОНИРОВАНИЕ, КОНТЕНТ-АДРЕСУЕМОЕ ХРАНИЛИЩЕ, КОНТРОЛЬ ВЕРСИЙ, РАЗРЕШЕНИЕ КОНФЛИКТОВ, КОНТЕЙНЕРИЗАЦИЯ, DOCKER, ASP.NET CORE, REST API, УЧЕБНЫЙ ПРОЦЕСС, МЕТАДААННЫЕ, СНИМКИ СОСТОЯНИЯ, ТРЁХСТОРОННЕЕ СЛИЯНИЕ, SHA-256, POSTGRESQL, OPENAPI.

Целью работы является разработка прототипа системы синхронизации файлов для системы поддержки учебного процесса, обеспечивающей распределённое хранение неструктурированных данных, версионирование объектов и корректное разрешение конфликтов при одновременном изменении файлов на разных узлах.

Для достижения цели выполнен анализ возможностей платформы .NET для манипуляции файловой системой и асинхронного взаимодействия, рассмотрены подходы к децентрализованному хранению неструктурированных данных, проведён сравнительный анализ систем контроля версий Git и SVN, разработаны модели метаданных и алгоритмы синхронизации, спроектирована архитектура системы и программные интерфейсы взаимодействия узлов.

В первом разделе проводится анализ возможностей платформы .NET для работы с файловой системой и асинхронного взаимодействия, рассматриваются децентрализованные архитектуры хранения и сравниваются подходы к версионированию объектов. Во втором разделе разрабатываются модели метаданных для неструктурированных и структурированных данных, формализуются алгоритмы синхронизации версий и механизмы разрешения конфликтов на основе трёхстороннего сравнения хеш-сумм. В третьем разделе проектируется архитектура контейнеризированной системы хранения, схема базы данных PostgreSQL и REST API для взаимодействия узлов синхронизации. В четвёртом разделе описывается программный прототип, реализованный на платформе .NET 10 с использованием ASP.NET Core, Entity Framework Core и Docker; приводятся результаты отладки, функционального тестирования через Swagger UI и браузерный графический интерфейс, а также наполнения тестовыми учебными данными.

Содержание

Реферат.....	2
Содержание.....	3
Введение.....	4
1 Анализ возможностей платформы .NET и подходов к распределённому хранению данных.....	6
1.1 Анализ возможностей платформы .NET для манипуляции файловой системой с точки зрения создания виртуальных папок.....	6
1.2 Анализ возможностей платформы .NET для разработки систем распределённого хранения на основе асинхронного взаимодействия.....	9
1.3 Анализ возможностей компонентной разработки пользовательских интерфейсов веб- и мобильных приложений.....	13
1.4 Сравнительный анализ подходов к версионированию объектов (Git, SVN и аналоги).....	15
2 Моделирование предметной области и версионирования файлов.....	18
2.1 Разработка моделей метаданных для хранения и версионирования неструктурированных данных.....	18
2.2 Разработка моделей объектов данных и метаданных для версионирования структурированных объектов.....	22
2.3 Разработка алгоритмов синхронизации версий в распределённой среде.....	24
3 Проектирование интерфейсных решений и архитектуры приложения.....	32
3.1 Проектирование архитектуры децентрализованной системы хранения неструктурированных данных.....	32
Обоснование выбора технологий.....	32
3.2 Проектирование фрагмента схемы базы данных для описания версий на разных узлах..	36
3.3 Проектирование программных интерфейсов (API) взаимодействия узлов синхронизации.....	37
4 Реализация прототипа, тестирование и наполнение данными.....	41
4.1 Разработка программного прототипа системы распределённого хранения и синхронизации.....	41
4.2 Механизмы загрузки данных и наполнение тестовыми файлами.....	41
4.3 Отладка, функциональное тестирование и оценка производительности.....	42
Заключение.....	45
Список литературы.....	47
Приложение А. Демонстрация процесса разрешения конфликтов.....	49

Введение

Современная образовательная среда постепенно переходит от локальных файловых хранилищ и ручного обмена документами к комплексным распределённым системам поддержки учебного процесса. В таких системах студенты, преподаватели и администраторы работают с учебными материалами, методическими указаниями, лабораторными работами и отчётами, которые должны быть доступны с любого устройства, синхронизированы между участниками и защищены от потери данных

Особенно заметной становится потребность в механизмах синхронизации файлов. Пользователь учебной системы часто работает на разных устройствах: создаёт документы на стационарном компьютере в аудитории, редактирует их на ноутбуке дома, просматривает на планшете в библиотеке. Преподавателю требуется обновить методические материалы, загрузить новые задания, получить отчёты от студентов. Студенту важно иметь актуальную версию файлов, видеть историю изменений и не потерять свои наработки при одновременной работе над одним документом.

Основная проблема состоит в том, что при распределённом хранении файлов возникают конфликты версий. Когда два пользователя одновременно изменяют один и тот же файл, система должна либо автоматически объединить изменения (если это возможно), либо корректно уведомить о конфликте и предоставить механизмы его разрешения. При прямом копировании файлов между узлами пользователь получает противоречивые версии, теряет данные или не понимает, какая версия является актуальной. Это приводит к росту числа действий, снижению надёжности и увеличению вероятности потери информации.

Новизна работы заключается в том, что система синхронизации рассматривается не просто как механизм копирования файлов, а как распределённая система с версионированием объектов и интеллектуальным разрешением конфликтов. Для отслеживания изменений вводятся метаданные файлов: хеш-суммы содержимого, временные метки, авторы изменений. Такая модель позволяет строить механизмы синхронизации не по принципу «последний записал — тот и прав», а с учётом истории изменений и контекста работы пользователей.

Оригинальная суть исследования состоит в объединении трёх линий: теории распределённых систем и децентрализованного хранения данных, механизмов версионирования объектов (по аналогии с системами контроля версий Git и SVN) и практического прототипа системы синхронизации на платформе .NET. В качестве программной базы используется прототип, реализующий локальный и удалённый узлы синхронизации, механизмы сравнения версий, автоматического объединения изменений и ручного разрешения конфликтов.

Целью учебно-исследовательской работы является разработка прототипа системы синхронизации файлов для системы поддержки учебного процесса, обеспечивающей распределённое хранение неструктурированных данных, версионирование объектов и корректное разрешение конфликтов при одновременном изменении файлов на разных узлах.

Нормативная база оформления и терминологии работы включает методические указания кафедры к пояснительным запискам УИР/НИР, стандарты ЕСПД по видам программных документов и стадиям разработки, стандарт на схемы алгоритмов, программ, данных и систем, а также стандарты терминов в области программного обеспечения и систем обработки информации [1-6].

В первом разделе проводится анализ возможностей платформы .NET для манипуляции файловой системой и асинхронного взаимодействия, анализ децентрализованных архитектур и подходов к версионированию объектов. Во втором разделе разрабатываются модели метаданных для хранения неструктурированных данных, алгоритмы синхронизации версий и механизмы разрешения конфликтов. В третьем разделе проектируется архитектура децентрализованной системы хранения и схема базы данных для описания версий на разных узлах. В четвёртом разделе описывается программный прототип системы синхронизации, механизмы загрузки данных, отладка, тестирование и оценка производительности.

1 Анализ возможностей платформы .NET и подходов к распределённому хранению данных

В разделе рассматриваются технологические и методические основания разработки прототипа системы синхронизации файлов для системы поддержки учебного процесса. Анализ проводится не только по критериям программной реализации, но и по критериям пригодности для систем распределённого хранения с насыщенной логикой версионирования: манипуляции файловой системой, асинхронного взаимодействия узлов, децентрализованных архитектур, разрешения конфликтов при одновременном изменении объектов и сравнения подходов к контролю версий (Git, SVN и аналоги).

1.1 Анализ возможностей платформы .NET для манипуляции файловой системой с точки зрения создания виртуальных папок.

Платформа .NET предоставляет развитый набор средств для работы с файловой системой, которые образуют фундамент для построения системы синхронизации файлов. Базовое пространство имён System.IO содержит классы File, Directory, FileInfo, DirectoryInfo, Path и FileStream, покрывающие операции чтения, записи, перечисления, копирования, перемещения и удаления файлов и каталогов [7]. Эти средства являются синхронными по умолчанию, однако платформа предоставляет и асинхронные аналоги (File.ReadAllBytesAsync, FileStream.CopyToAsync, StreamReader.ReadLineAsync), что критично для системы синхронизации, где операции ввода-вывода не должны блокировать пользовательский интерфейс или поток обработки сетевых запросов [8].

Для системы синхронизации файлов понятие «виртуальной папки» имеет особое значение. Виртуальная папка — это не обязательно физический каталог на диске, а логическое представление набора файлов, которое может отображать данные из распределённого хранилища, кэша или метаданных. В контексте учебной системы виртуальная папка может соответствовать, например, «материалам курса», «лабораторным работам группы» или «отчётам студента», причём физически эти файлы могут быть разбросаны по разным узлам хранения.

Платформа .NET поддерживает несколько подходов к организации виртуальных папок и хранению загружаемых пользователем файлов.

Прямая работа с файловой системой хоста. Классы System.IO позволяют создавать, читать и изменять реальные каталоги на локальном диске в произвольном месте (например, в Documents пользователя). Этот подход прост в реализации, но создаёт серьёзные риски: пользователь может случайно удалить, переместить или переименовать файлы вне системы синхронизации, что нарушит целостность версий и приведёт к рассогласованию метаданных. Для системы синхронизации такой подход ненадёжен как основное хранилище.

Прямая работа с файловой системой хоста. Классы System.IO позволяют создавать, читать и изменять реальные каталоги на локальном диске в произвольном месте (например, в Documents пользователя). Этот подход прост в реализации, но создаёт серьёзные риски: пользователь может случайно удалить, переместить или переименовать файлы вне системы синхронизации, что нарушит целостность версий и приведёт к рассогласованию метаданных. Для системы синхронизации такой подход ненадёжен как основное хранилище.

Виртуальная файловая система (VFS). Более сложный подход предполагает реализацию собственного уровня абстракции, который перехватывает операции с файлами и направляет их в распределённое хранилище. В Windows для этого можно использовать Minifilter-драйверы, в Unix - FUSE (Filesystem in Userspace). Платформа .NET не предоставляет встроенных средств для создания VFS, но может взаимодействовать с такими системами через стандартные API ввода-вывода. Этот подход используется в промышленных системах синхронизации (например, Dropbox, OneDrive), но требует значительных усилий по реализации и избыточен для прототипа.

Локальное изолированное хранилище приложения. Наиболее практичный и надёжный подход — копирование загружаемых файлов в локальное хранилище, принадлежащее исключительно приложению. Это может быть выделенная директория внутри контейнера (например, Docker-volume), служебная папка в AppData или изолированное хранилище на серверной части. Пользователь взаимодействует с файлами только через интерфейс приложения (например, через кнопку «Add file»), а физическое содержимое находится вне зоны прямого доступа из файлового менеджера ОС. Такой подход гарантирует, что файлы не будут случайно удалены или изменены пользователем, а система синхронизации сохраняет полный контроль над версиями и метаданными. Как отмечается в работе Kleppmann M., надёжные системы хранения данных должны обеспечивать изоляцию данных от внешних воздействий и централизованное управление версиями [9].

Таблица 1.1 — Сравнение подходов к организации хранения файлов в системе синхронизации

Подход	Основная идея	Преимущества	Недостатки	Пригодность для УИР
Прямая работа с System.IO в файловой системе хоста	Реальные каталоги в произвольном месте диска	Простота, прямой доступ	Риск случайного удаления, нарушение версионирования	Низкая для основной системы
Символьные ссылки	Перенаправление на другой каталог	Прозрачность для приложений	Только локальная файловая система, требуются права	Низкая для распределённой системы

Виртуальная файловая система (VFS)	Перехват операций на уровне ОС	Полная прозрачность	Требует драйверов, сложно в реализации	Избыточно для прототипа
Локальное изолированное хранилище приложения	Копирование файлов в защищённое хранилище (Docker-volume, AppData)	Защита от удаления, целостность версий, переносимость	Требует UI для взаимодействия	Высокая, оптимально для УИР

Для данной УИР наиболее подходящим является подход с локальным изолированным хранилищем приложения. Файлы, загружаемые пользователем через интерфейс (кнопка «Add file»), копируются в защищённое хранилище, принадлежащее исключительно backend-части системы. Это решение обладает рядом критических преимуществ:

1. Защита от случайного удаления. Поскольку файлы находятся в изолированном хранилище (например, в Docker-volume или служебной директории), пользователь не может случайно удалить их через проводник или терминал. Все операции с файлами проходят через backend, что гарантирует согласованность метаданных и физического содержимого.
2. Целостность версионирования. Система синхронизации отслеживает версии файлов через хеш-суммы и метаданные. Если пользователь может напрямую изменять файлы в хранилище, он может нарушить связь между версией в базе данных и физическим файлом. Изолированное хранилище исключает эту возможность: все операции (чтение, запись, удаление) проходят через backend, который гарантирует согласованность.
3. Переносимость и восстановление. Если пользователь удалит приложение с хост-машины, данные в изолированном хранилище (Docker-volume) сохранятся. При повторном развёртывании контейнера с подключением того же volume все файлы и версии будут восстановлены. Это соответствует принципу устойчивости распределённых систем, описанному в работе Tanenbaum A. S., Van Steen M., где подчёркивается важность отделения состояния приложения от его выполнения [10].
4. Контроль доступа. Только backend имеет прямой доступ к файлам, что позволяет централизованно управлять правами, логировать операции и применять бизнес-правила (например, проверку дубликатов по хешу).

Важным аспектом является асинхронная природа операций с файлами. Класс FileStream поддерживает асинхронное чтение и запись через методы ReadAsync, WriteAsync, CopyToAsync, что позволяет не блокировать поток при работе с большими файлами [8]. Для системы синхронизации это критично, так как одновременная загрузка нескольких файлов с

удалённого узла не должна приводить к «зависанию» интерфейса. Использование паттерна `async/await` в C# позволяет писать асинхронный код, который выглядит как синхронный, что снижает сложность разработки [11].

Для вычисления хеш-сумм файлов (что необходимо для отслеживания изменений и версионирования) платформа .NET предоставляет классы из пространства имён `System.Security.Cryptography`: SHA256, MD5, SHA1. Метод `ComputeHash` принимает поток данных и возвращает хеш, что позволяет вычислять контрольные суммы даже для очень больших файлов без загрузки их целиком в память [7].

Таким образом, платформа .NET предоставляет все необходимые средства для реализации системы синхронизации файлов: асинхронный ввод-вывод, работу с файловой системой, криптографические хеш-функции. Оптимальным подходом к организации хранения является копирование загружаемых файлов в локальное изолированное хранилище приложения, что обеспечивает защиту данных от случайного удаления пользователем, целостность версионирования и переносимость системы. При этом архитектура должна быть спроектирована так, чтобы в дальнейшем слой хранения мог быть заменён на полноценную VFS без изменения бизнес-логики [12].

1.2 Анализ возможностей платформы .NET для разработки систем распределённого хранения на основе асинхронного взаимодействия.

Распределённое хранение файлов требует постоянного обмена данными между узлами: локальным клиентом, серверным backend и, потенциально, другими узлами синхронизации. Такой обмен характеризуется высокой задержкой сети, непредсказуемым временем отклика и необходимостью одновременной обработки множества запросов. В этих условиях синхронный подход к вводу-выводу становится неприемлемым: он блокирует потоки, снижает пропускную способность системы и ухудшает отзывчивость интерфейса. Платформа .NET предоставляет развитый набор средств для построения асинхронных распределённых систем, которые образуют технологическую основу прототипа системы синхронизации.

Фундаментальной основой асинхронного программирования в .NET является паттерн `async/await`, реализованный через класс `Task` и `Task<T>` из пространства имён `System.Threading.Tasks`. Как отмечается в работе Richter J. «CLR via C#», этот подход позволяет писать асинхронный код, который читается как синхронный, при этом компилятор автоматически разворачивает его в машину состояний, не блокирующую потоки при ожидании операций ввода-вывода [8]. Это критично для системы синхронизации, где операции чтения файлов, вычисления хешей и сетевых запросов выполняются параллельно.

Класс `Task` представляет собой обещание (promise) завершения операции в будущем. Методы `Task.WhenAll`, `Task.WhenAny` позволяют организовать параллельное выполнение

нескольких задач, что необходимо, например, при одновременной загрузке нескольких файлов с удалённого узла или при параллельном вычислении хеш-сумм для множества файлов в директории. Albahari J. в справочнике «C# 12 in a Nutshell» подчёркивает, что правильное использование `async/await` позволяет достичь высокой пропускной способности системы при минимальном числе потоков, что особенно важно для серверных приложений, обрабатывающих множество одновременных подключений [11].

Важным аспектом является разделение CPU-bound и I/O-bound операций. Вычисление хеш-сумм файлов является CPU-bound задачей и должно выполняться в отдельном потоке через `Task.Run`, тогда как чтение файлов и сетевые запросы являются I/O-bound и не требуют выделения потока на время ожидания. Такое разделение позволяет оптимизировать использование ресурсов системы.

Платформа .NET предоставляет несколько уровней абстракции для организации сетевого взаимодействия между узлами синхронизации:

HttpClient и REST API. Класс `HttpClient` из пространства имён `System.Net.Http` является основным средством для взаимодействия с удалёнными узлами по протоколу HTTP. Он поддерживает асинхронные методы `GetAsync`, `PostAsync`, `PutAsync`, `DeleteAsync`, а также работу с потоками через `HttpContent.ReadAsStreamAsync`, что позволяет загружать и отправлять большие файлы без загрузки их целиком в память. REST API, построенный на принципах, описанных Fielding R. T. в его фундаментальной диссертации, является оптимальным стилем архитектуры для взаимодействия узлов синхронизации: он обеспечивает stateless-взаимодействие, единый интерфейс к ресурсам и возможность кэширования ответов [13].

ASP.NET Core для серверной части. Фреймворк ASP.NET Core используется для построения backend-части системы синхронизации, работающей в Docker-контейнере. Он предоставляет встроенную поддержку асинхронной обработки запросов, middleware для аутентификации и логирования, а также средства для работы с файлами через `IFormFile` и `Stream`. Freeman A. в руководстве «Pro ASP.NET Core 6» отмечает, что ASP.NET Core построен на полностью асинхронном конвейере обработки запросов, что позволяет серверу обрабатывать тысячи одновременных подключений при минимальных затратах ресурсов [14].

gRPC для высокопроизводительного взаимодействия. Для сценариев, требующих минимальной задержки и эффективной сериализации бинарных данных (например, передача дельт между версиями файлов), платформа .NET поддерживает gRPC — высокопроизводительный фреймворк на основе HTTP/2 и Protocol Buffers. Однако для прототипа системы синхронизации достаточно REST API, так как он проще в реализации и отладке.

SignalR для real-time уведомлений. Библиотека SignalR позволяет организовать двустороннюю связь между узлами в реальном времени, что может быть использовано для уведомления о новых версиях файлов или изменениях в метаданных. Для прототипа этот механизм избыточен, но архитектура системы должна допускать его подключение в дальнейшем.

Архитектура взаимодействия узлов синхронизации строится по клиент-серверной модели, где backend в Docker-контейнере выступает в роли сервера, а браузерный интерфейс и, потенциально, другие узлы — в роли клиентов. Калмыков А. Ю. отмечает, что клиент-серверная архитектура обеспечивает централизованное управление данными, безопасность и масштабируемость, что критично для системы синхронизации файлов [15].

Для организации асинхронного обмена данными между узлами в распределённых системах применяются различные паттерны. Kleppmann M. в работе «Designing Data-Intensive Applications» выделяет несколько ключевых подходов: синхронное взаимодействие (request-response), асинхронное взаимодействие через очереди сообщений и event-driven архитектуру [9]. Для прототипа системы синхронизации выбран подход request-response через REST API, так как он прост в реализации и отладке, а объём данных не требует использования очередей сообщений.

Tanenbaum A. S. и Van Steen M. в учебнике «Distributed Systems» подчёркивают важность обработки отказов и таймаутов при асинхронном взаимодействии [10].

Newman S. в работе «Building Microservices» отмечает, что при проектировании распределённых систем необходимо учитывать сетевую ненадёжность и закладывать механизмы graceful degradation, то есть корректной деградации функциональности при недоступности удалённых узлов [16]. Для системы синхронизации это означает, что при недоступности сервера пользователь может продолжать работать с локальным кэшем файлов, а синхронизация будет выполнена при восстановлении соединения.

Richardson C. в работе «Microservices Patterns» описывает паттерны Saga и CQRS для управления распределёнными транзакциями и разделения операций чтения и записи [17]. Для прототипа системы синхронизации паттерн Saga может быть применён для обеспечения согласованности при синхронизации: если загрузка файла на сервер прошла успешно, но обновление метаданных не удалось, система должна откатить изменения или пометить операцию как требующую разрешения.

Таблица 1.2 — Сравнение подходов к асинхронному взаимодействию в .NET

Подход	Основная идея	Преимущества	Недостатки	Пригодность для
--------	---------------	--------------	------------	-----------------

				УИР
Синхронный HTTP	Блокирующий вызов	Простота реализации	Блокировка потоков, низкая пропускная способность	Низкая
Асинхронный HTTP (HttpClient + async/await)	Неблокирующие запросы	Высокая пропускная способность, отзывчивость	Требуется понимания асинхронного программирования	Высокая, оптимально для УИР
gRPC	Бинарный протокол поверх HTTP/2	Минимальная задержка, эффективная сериализация	Сложнее в отладке, требует .proto-файлов	Избыточно для прототипа
SignalR (WebSockets)	Двусторонняя связь в реальном времени	Мгновенные уведомления	Избыточно для request-response сценариев	Перспективно для развития
Очереди сообщений (RabbitMQ, Kafka)	Асинхронный обмен через брокера	Надёжность, масштабируемость	Требуется дополнительной инфраструктуры	Избыточно для прототипа

Для данной УИР наиболее подходящим является подход с асинхронным HTTP через HttpClient и REST API. Он обеспечивает баланс между простотой реализации, производительностью и надёжностью, что критично для прототипа системы синхронизации. При этом архитектура системы должна быть спроектирована так, чтобы в дальнейшем можно было подключить gRPC для высокопроизводительных сценариев или SignalR для real-time уведомлений без изменения логики.

Важным аспектом асинхронного взаимодействия является обработка конкурентных операций, когда несколько узлов одновременно пытаются изменить один и тот же файл. Платформа .NET предоставляет примитивы синхронизации: SemaphoreSlim, lock, ConcurrentDictionary, которые позволяют организовать безопасный доступ к общим ресурсам. Для системы синхронизации это критично, так как необходимо гарантировать, что две одновременные загрузки файла не приведут к повреждению данных или потере версий.

Richter J. подчёркивает, что при работе с асинхронным кодом следует избегать использования lock внутри async-методов, так как это может привести к deadlock-ситуациям. Вместо этого рекомендуется использовать SemaphoreSlim с асинхронным методом WaitAsync, что позволяет организовать безопасный доступ к ресурсам без блокировки потоков [8].

1.3 Анализ возможностей компонентной разработки пользовательских интерфейсов веб- и мобильных приложений.

Неструктурированные данные (файлы, документы, изображения) составляют основу учебного процесса: методические материалы, лабораторные работы, отчёты и презентации. Традиционные централизованные файловые хранилища (FTP-серверы, общие папки) обладают единой точкой отказа, не обеспечивают встроенного версионирования и часто становятся «узким горлышком» при одновременной работе множества пользователей. Децентрализованные архитектуры предлагают решение этих проблем за счёт распределения данных между узлами, репликации и использования контент-адресуемого хранения.

В децентрализованной системе каждый узел может выступать как в роли клиента, так и в роли сервера, а данные хранятся в нескольких копиях. Это повышает отказоустойчивость и доступность. Kleppmann М. отмечает, что ключевой проблемой распределённых систем является обеспечение согласованности данных в условиях сетевых задержек и возможных разделений (partition) [1]. Для неструктурированных данных это означает, что два узла могут независимо изменить один и тот же файл, что требует механизмов обнаружения и разрешения конфликтов.

Существует несколько подходов к децентрализованному хранению неструктурированных данных:

Контент-адресуемое хранилище (Content-Addressable Storage, CAS). Файлы идентифицируются не по имени или пути, а по хеш-сумме их содержимого. Если содержимое меняется, меняется и его идентификатор. Этот подход используется в Git и IPFS. Он гарантирует целостность данных (любая модификация файла будет обнаружена при проверке хеша) и обеспечивает дедупликацию (одинаковые файлы хранятся только один раз).

Одноранговые сети (P2P) и распределённые хеш-таблицы (DHT). Узлы обмениваются файлами напрямую, минуя центральный сервер. Протоколы вроде BitTorrent эффективно распределяют нагрузку по загрузке больших файлов. Однако для учебной системы, где важна управляемость, аудит и контроль доступа, чистый P2P-подход может быть избыточным и сложным в реализации.

Распределённые системы контроля версий (DVCS). Системы вроде Git являются классическим примером децентрализованного хранилища. Каждый узел хранит полную историю изменений (граф коммитов). Синхронизация происходит через обмен дельтами (изменениями) и слияние (merge) графов. Этот подход идеально подходит для файлов, которые часто редактируются, но требует сложной логики для слияния бинарных файлов.

Объектные хранилища с репликацией. Хотя такие системы (например, MinIO, Ceph) часто имеют централизованный интерфейс, внутри они используют децентрализованные

алгоритмы кворумов и репликации для обеспечения долговечности данных. Они предоставляют простой API для работы с файлами как с неизменяемыми объектами (blobs).

Таблица 1.3 - Сравнение подходов к децентрализованному хранению неструктурированных данных

Подход	Основная идея	Преимущества	Ограничения	Пригодность для УИР
Контент-адресуемое хранилище (CAS)	Идентификация по хешу, Merkle-деревья	Целостность, дедупликация, версионирование	Требует вычисления хешей, неочевидно для пользователя	Высокая (основа версионирования)
P2P-сети (BitTorrent, IPFS)	Прямой обмен между узлами	Масштабируемость, отказоустойчивость	Сложность управления доступом, NAT-проблемы	Низкая для прототипа
DVCS (Git)	Граф коммитов, полная история на каждом узле	Мощный merge, аудит истории	Конфликты слияния бинарных файлов	Высокая (модель конфликтов)
Объектные хранилища (S3, MinIO)	Хранение по ключам, репликация	Простота API, масштабируемость	Требует инфраструктуры	Средняя (как удалённый узел)

Проведённый сравнительный анализ систем контроля версий показывает, что выбор модели версионирования критически влияет на архитектуру распределённой системы синхронизации, её отказоустойчивость и механизмы разрешения коллизий.

Централизованные системы (SVN, CVS) опираются на единый репозиторий и пессимистичные блокировки, что создаёт единую точку отказа и требует постоянного наличия сетевого соединения с сервером [18]. Для системы поддержки учебного процесса, где пользователи (студенты и преподаватели) могут работать автономно и синхронизироваться эпизодически, такой подход является ограничивающим фактором.

Децентрализованные системы (Git, Mercurial) предлагают принципиально иную парадигму. Как отмечается в фундаментальном руководстве Chacon S., Straub B. [19], Git не хранит списки изменений файлов (дельты), а работает со снимками (snapshots) файловой системы на каждом этапе. Каждый узел системы обладает полной историей изменений, что обеспечивает высокую живучесть и возможность локальной работы без обращения к центральному серверу [10].

Для разрабатываемого прототипа системы синхронизации файлов не требуется полный граф коммитов (DAG), используемый в Git, поскольку система не поддерживает ветвление и сложное слияние нескольких линий разработки. Однако принципы Git являются концептуальной основой для проектируемой модели

1.4 Сравнительный анализ подходов к версионированию объектов (Git, SVN и аналоги).

Системы контроля версий (Version Control Systems, VCS) являются фундаментальным инструментом для управления изменениями файлов в распределённой среде. Для системы синхронизации файлов учебного процесса выбор модели версионирования определяет архитектуру хранения, механизмы обнаружения конфликтов и способы их разрешения. В данном подразделе проводится сравнительный анализ двух наиболее распространённых подходов: централизованной системы SVN (Apache Subversion) и распределённой системы Git, с целью выявления оптимальной модели для проектируемого прототипа.

Git является мощной системой контроля версий, но для задачи синхронизации файлов учебного процесса ряд её возможностей является избыточным:

1. Полный граф коммитов (DAG - Directed Acyclic Graph). Git хранит историю в виде ориентированного ациклического графа, где каждый коммит указывает на одного или нескольких родителей. Это необходимо для поддержки сложного ветвления, слияния нескольких линий разработки и переписывания истории. Для системы синхронизации файлов учебного процесса достаточно линейной истории версий для каждого файла: каждая версия указывает только на одну предыдущую версию. Это упрощает модель хранения и логику синхронизации.
2. Локальное ветвление. Git позволяет создавать лёгкие ветки для параллельной разработки. В учебном процессе студент работает над одним файлом (например, лабораторной работой), и ему не нужно создавать параллельные линии разработки. Достаточно одной "локальной ветки" (текущей рабочей версии) и одной "серверной ветки" (мастер-версии).
3. Переписывание истории (rebase, reset, revert). Эти операции позволяют изменять историю коммитов, что полезно для очистки истории перед публикацией. Для системы синхронизации файлов это не требуется: история изменений должна быть неизменяемой для обеспечения аудита и целостности данных.
4. Подмодули и вложенные репозитории. Git поддерживает вложенные репозитории, что полезно для сложных проектов с зависимостями. Для учебного процесса это избыточно: файлы хранятся в плоской структуре или с простой иерархией папок.

5. Хуки (hooks) и автоматизация. Git поддерживает скрипты, выполняемые при определённых событиях (pre-commit, post-commit). Для прототипа системы синхронизации это не требуется.
6. Поддержка тегов и релизов. Git позволяет пометить определённые коммиты тегами (например, v1.0, v2.0). Для учебного процесса это не требуется: версии файлов идентифицируются по хешам и временным меткам.

Таким образом, для УИР достаточно использовать упрощённую модель Git: контент-адресуемое хранилище снимков (snapshots) с линейной историей версий и механизмом трёхстороннего слияния.

SVN, несмотря на централизованную архитектуру, предлагает ряд удобных механизмов, которые стоит заимствовать:

1. Механизм разрешения конфликтов через три файла. При обнаружении конфликта SVN создаёт три файла:
 - file.mine - локальная версия пользователя
 - file.r<server_rev> - серверная версия (актуальная на сервере)
 - file.r<base_rev> - базовая версия (общий предок, от которого оба отталкивались)
2. Простая линейная история. SVN хранит историю в виде последовательности ревизий (номеров), где каждая ревизия - это дельта (изменение) относительно предыдущей. Для системы синхронизации это удобно: история каждого файла - это просто список версий, упорядоченных по времени.

На основе проведённого анализа для системы синхронизации файлов учебного процесса принимается гибридная модель, сочетающая лучшие черты Git и SVN:

От Git заимствуется:

1. Локальная история версий. Каждый узел (локальный кэш пользователя) хранит полную историю изменений файлов. Это позволяет работать офлайн, просматривать историю и откатывать изменения.
2. Контент-адресуемое хранилище. Каждая версия файла идентифицируется по хеш-сумме (SHA-256). Это гарантирует целостность данных, обеспечивает дедупликацию и упрощает сравнение версий.
3. Модель снимков (snapshots). Вместо хранения дельт система хранит полные снимки файлов. Это упрощает логику синхронизации и разрешения конфликтов.

От SVN заимствуется:

1. Механизм разрешения конфликтов через три файла. При обнаружении конфликта система создаёт три файла: локальную версию, серверную версию и базовую версию

(общий предок). Это особенно важно для бинарных файлов (Excel, Word, PDF), которые невозможно автоматически объединить.

2. Линейная история версий. Вместо полного графа коммитов (DAG) используется упрощённая модель: каждая версия файла указывает только на одну предыдущую версию. Этого достаточно для сценариев учебного процесса.

1.5 Выводы по аналитической части.

Платформа .NET предоставляет исчерпывающий набор асинхронных средств ввода-вывода и криптографического хеширования. Для системы синхронизации оптимальным является отказ от прямого доступа к файлам в файловой системе хоста в пользу локального изолированного хранилища (например, Docker-volume) и слоя метаданных. Это гарантирует защиту данных от случайного удаления пользователем, обеспечивает целостность версионирования и переносимость системы.

Использование паттерна `async/await`, `HttpClient` и REST API позволяет построить неблокирующее взаимодействие между узлами синхронизации. Это критично для обработки больших учебных файлов и обеспечения отзывчивости браузерного интерфейса при одновременной загрузке и вычислении хеш-сумм.

Для неструктурированных данных учебного процесса (документы, отчёты, презентации) наиболее подходящим является контент-адресуемое хранилище (CAS), работающее со снимками (snapshots) файлов. Такой подход обеспечивает естественную дедупликацию, гарантию целостности данных и упрощает алгоритмы сравнения версий.

Прямое использование централизованной модели SVN ограничивает автономную работу пользователей, а полный граф коммитов (DAG) Git является избыточным для задач синхронизации файлов. Принимается модель, наследующая от Git контент-адресуемость, локальную линейную и неизменяемую историю версий и снимки, а от SVN — механизм создания трёх файлов (базовая, локальная и серверная версии) для ручного разрешения конфликтов. Это особенно важно для бинарных файлов (Excel, Word, PDF), широко используемых в учебном процессе и не поддающихся автоматическому текстовому слиянию. Механизмы откатов (`reset`, `rebase`) исключены из модели для обеспечения прозрачности аудита и устойчивости синхронизации.

2 Моделирование предметной области и версионирования файлов.

Во втором разделе описывается модель предметной области системы синхронизации файлов и переход от концептуальных представлений к формализованным структурам данных. Основная идея состоит в том, что система синхронизации не должна механически копировать файлы между узлами. Она должна опираться на контент-адресуемое хранилище, снимки состояния файловой системы и явный механизм обнаружения и разрешения конфликтов. В разделе разрабатываются модели метаданных для неструктурированных и структурированных данных, а также формализуются алгоритмы синхронизации и разрешения коллизий.

2.1 Разработка моделей метаданных для хранения и версионирования неструктурированных данных.

Неструктурированные данные в учебном процессе представлены файлами различных форматов: документами Word и PDF, таблицами Excel, презентациями PowerPoint, изображениями, архивами и исходным кодом. В отличие от структурированных данных (записей в базе данных, JSON-объектов), неструктурированные файлы не имеют фиксированной схемы, и их содержимое может быть произвольным бинарным или текстовым потоком. Для системы синхронизации это означает, что нельзя полагаться на семантический анализ содержимого файла для обнаружения изменений или слияния версий. Вместо этого система должна опираться на метаданные - формализованное описание состояния файловой системы, её снимков и контекста синхронизации.

Ключевым принципом модели является контент-адресуемость: каждая версия файла идентифицируется не по имени или пути, а по хеш-сумме её содержимого. Как отмечается в руководстве Chacon S., Straub B. по Git, такой подход гарантирует, что любое изменение содержимого файла приводит к изменению его идентификатора, а одинаковые файлы (даже с разными именами) получают один и тот же идентификатор [19]. Это обеспечивает три критических свойства:

1. Целостность данных. Если содержимое файла будет повреждено при передаче или хранении, вычисленный хеш не совпадёт с сохранённым, и система немедленно обнаружит ошибку.
2. Дедупликация. Если два пользователя загрузят одинаковый файл (например, шаблон лабораторной работы), система сохранит только одну физическую копию, а в метаданных создаст две ссылки на один и тот же хеш.
3. Простота сравнения версий. Для определения, изменился ли файл, достаточно сравнить хеши, а не побайтно анализировать содержимое.

Физическое хранилище организуется как набор blob-объектов, имена которых совпадают с их хешами. Каждый blob представляет собой неизменяемый (immutable) объект: после сохранения его содержимое не может быть изменено, а любая модификация файла приводит к созданию нового blob с новым хешем. Такая модель исключает коллизии имён и упрощает репликацию между узлами: для передачи файла достаточно отправить его хеш и содержимое, а принимающий узел сохраняет его под именем, равным хешу.

Вместо хранения отдельной истории для каждого файла система оперирует состоянием машины - снимком (snapshot) всей файловой системы на определённый момент времени. Состояние машины формализуется как коммит, содержащий:

- Идентификатор коммита - уникальный UUID, однозначно определяющий этот снимок;
- Временная метка - дата и время создания снимка;
- Автор - идентификатор пользователя, создавшего снимок;
- Комментарий - опциональное описание изменений;
- Словарь файлов - отображение "путь+имя файла → хеш blob".

Таким образом, коммит не хранит физическое содержимое файлов, а лишь ссылки на уникальные blob-объекты в контент-адресуемом хранилище. История файла выводится из последовательности коммитов: для получения истории файла система просматривает все коммиты в порядке от нового к старому и извлекает хеши, связанные с этим путём+именем. Такой подход устраняет дублирование метаданных и гарантирует согласованность: история файла всегда соответствует истории машины.

Таблица 2.1 — Атрибуты модели коммита

Атрибут	Тип данных	Назначение	Пример
commit_id	UUID	Уникальный идентификатор снимка	550e8400-e29b-41d4-a716-446655440000
timestamp	datetime	Время создания снимка	2026-03-20 14:30:00
author	UUID	Идентификатор пользователя	user_123
comment	string	Описание изменений	"Sync with server"
files	dictionary	Отображение путь+имя → хеш blob	{"courses/math/file1.docx": "a1b2c3..."}

Для корректного обнаружения конфликтов клиент хранит снимок последнего синхронизированного состояния сервера (last_sync_snapshot). Этот снимок представляет собой словарь "путь+имя файла → хеш blob", зафиксированный на момент последней успешной синхронизации. При следующей синхронизации клиент сравнивает три хеша для каждого файла: хеш из снимка синхронизации (base_hash), локальный хеш (local_hash) и

серверный хеш (server_hash). Это позволяет точно определить, кто и что изменил файл, и избежать ложных конфликтов.

Таблица 2.2 — Атрибуты модели снимка синхронизации.

Атрибут	Тип данных	Назначение	Пример
snapshot_timestamp	datetime	Время последнего снимка	2026-03-19 10:00:00
files	dictionary	Отображение путь+имя → хеш blob	{"courses/math/file1.docx": "a1b2c3..."}
server_commit_id	UUID	Идентификатор коммита сервера	770e8400-e29b-41d4-a716-446655440001

Для каждого файла в системе определяется его состояние синхронизации, которое классифицируется на основе сравнения трёх хешей: base_hash (из снимка синхронизации), local_hash (из текущего локального состояния) и server_hash (из текущего состояния сервера). Модель состояния файла формализуется следующим образом:

$FileState = \langle path, base_hash, local_hash, server_hash, status \rangle$

где:

- path - путь+имя файла (например, "/courses/math/file1.docx")
- base_hash - хеш файла в снимке синхронизации (или null, если файла не было при последней синхронизации).
- local_hash - хеш файла в текущем локальном состоянии (или null, если файл отсутствует локально)
- server_hash - хеш файла в текущем состоянии сервера (или null, если файл отсутствует на сервере)
- status - статус синхронизации, определяемый по правилам, приведённым в таблице 2.3

Таблица 2.3 - Классификация состояний файла.

Условие	Статус
local_hash == server_hash	synced
local_hash == base_hash, server_hash != base_hash	server_modified
local_hash != base_hash, server_hash == base_hash	local_modified
local_hash != base_hash, server_hash != base_hash, local_hash != server_hash	conflict
base_hash == null, local_hash != null, server_hash == null	new_local
base_hash == null, local_hash == null, server_hash != null	new_server
	independent_
base_hash == null, local_hash != null, server_hash != null	creation

Таблица 2.4 — Классификация состояний файла, и необходимое действие.

Статус	Действие
--------	----------

synced	Файл не требует изменений
server_modified	Принять серверную версию
local_modified	Отправить локальную версию на сервер
conflict	Разрешить конфликт вручную
new_local	Отправить новый файл на сервер
new_server	Принять новый файл локально
independent_creation	Разрешить конфликт независимого создания

При обнаружении статуса `conflict` или `independent_creation` система создаёт объект конфликта, содержащий ссылки на три blob-объекта: базовую версию (если существует), локальную версию и серверную версию. Модель конфликта формализуется следующим образом:

$$Conflict = \left(\begin{array}{l} path \\ base_blob_hash \\ local_blob_hash \\ server_blob_hash \\ resolved_blob_hash \\ resolved_by \\ resolved_at \end{array} \right)$$

где:

- `path` - путь+имя конфликтного файла.
- `base_blob_hash` - хеш базовой версии (или `null`, если общего предка нет).
- `local_blob_hash` - хеш локальной версии.
- `server_blob_hash` - хеш серверной версии.
- `resolved_blob_hash` - хеш разрешённой версии (после того, как пользователь объединил изменения).
- `resolved_by` - идентификатор пользователя, разрешившего конфликт.
- `resolved_at` - время разрешения конфликта.

Пользователь получает три файла: базовую версию (показывает, как файл выглядел до изменений), локальную версию (показывает, что изменил пользователь) и серверную версию (показывает, что изменил другой пользователь). Пользователь сравнивает эти три версии, выбирает нужную или объединяет изменения вручную, а затем сохраняет итоговую версию как новую. Система вычисляет хеш этой версии, создаёт новый blob и обновляет состояние файла.

Каждый узел системы (локальный клиент или сервер) хранит своё текущее состояние, историю коммитов и, в случае клиента, снимок последней синхронизации. Модель узла формализуется следующим образом:

$$Node = \left\{ \begin{array}{l} node_id \\ node_type \\ current_commit \\ commit_history \\ blob_storage \\ last_sync_snapshot \end{array} \right\}$$

где:

- `node_id` - уникальный идентификатор узла;
- `node_type` - тип узла (`local` - локальный клиент, `server` - удалённый сервер);
- `current_commit` - текущий коммит (снимок состояния файловой системы);
- `commit_history` - список коммитов в порядке от нового к старому;
- `blob_storage` - контент-адресуемое хранилище blob-объектов;
- `last_sync_snapshot` - снимок последнего синхронизированного состояния сервера (только для клиента).

Локальный узел хранит:

- Текущее состояние файловой системы (коммит);
- Историю коммитов (для отката и просмотра изменений);
- Физические blob-объекты для всех версий файлов;
- Снимок последней синхронизации (для обнаружения конфликтов).

Серверный узел хранит:

- Мастер-коммит (текущее актуальное состояние файловой системы);
- Историю коммитов;
- Физические blob-объекты для всех версий файлов.

2.2 Разработка моделей объектов данных и метаданных для версионирования структурированных объектов.

В контексте системы поддержки учебного процесса и синхронизации данных под структурированными объектами понимаются данные, имеющие внутреннюю семантическую схему: конфигурационные файлы системы (JSON, XML), таблицы успеваемости в формате CSV, манифесты курсов, а также метаданные, описывающие права доступа или настройки интерфейса. В отличие от неструктурированных бинарных файлов (документы Word, презентации, изображения), структурированные объекты теоретически допускают

семантический анализ, построчное или попольное сравнение и автоматическое слияние (merge) на уровне отдельных полей.

В современной инженерии данных для версионирования структурированных объектов применяются специализированные механизмы, позволяющие хранить не полные снимки, а только дельты (изменения) на уровне полей. К таким подходам относятся:

1. Алгоритмы структурного диффа (например, JSON Patch, RFC 6902). Система сравнивает два JSON-документа и генерирует последовательность операций (add, remove, replace), описывающих переход от старой версии к новой. Это позволяет экономить место и автоматически сливать изменения, если они затрагивают разные поля объекта.
2. Событийное хранение (Event Sourcing) и CDC (Change Data Capture). Каждое изменение структурированного объекта фиксируется как отдельное событие в журнале. Состояние объекта восстанавливается путем воспроизведения (replay) всех событий с начала времен.
3. Документо-ориентированные хранилища с ревизиями. Базы данных (например, CouchDB) автоматически хранят историю изменений документов, обеспечивая версионирование на уровне СУБД.

Как отмечается в фундаментальной работе Kleppmann M. по проектированию интенсивных данных, специализированные форматы кодирования и дифференциальное хранение позволяют достичь высокой эффективности при работе со структурированными схемами [9]. Fowler M. также подчеркивает важность разделения ответственности при работе с разными типами данных в корпоративных архитектурах [12].

Несмотря на теоретическую оптимальность построчного или попольного версионирования для структурированных данных, его внедрение в рамках проектируемого прототипа системы синхронизации файлов признано нецелесообразным по следующим причинам:

1. Нарушение архитектурного единообразия. Если для бинарных файлов (Excel, PDF) используется контент-адресуемое хранилище (CAS) и механизм ручного разрешения конфликтов через три файла, а для структурированных файлов (JSON, CSV) — алгоритмы автоматического слияния на уровне полей, система приобретает дуалистическую архитектуру. Этократно усложняет логику синхронизации, алгоритм поиска общего предка и механизм уведомления пользователя о конфликтах.

2. Сложность автоматического слияния в учебном контексте. Даже если файл имеет структуру (например, CSV-таблица с оценками), изменения в ней часто носят семантический характер (преподаватель исправил оценку студенту, студент добавил строку с новым заданием). Автоматический построчный merge в таких случаях может привести к скрытым логическим ошибкам и смещению колонок, что недопустимо для академических данных.
3. Избыточность для прототипа. Реализация собственного движка структурного диффа и трехстороннего слияния для JSON/XML выходит за рамки задачи разработки прототипа системы синхронизации файлов и смещает фокус с исследования механизмов распределенного хранения на разработку парсеров данных.

В связи с этим, в силу требований к обеспечению строгого единообразия (унификации) механизма версионирования, принимается ключевое проектное решение: рассматривать любые данные - как неструктурированные бинарные файлы, так и структурированные объекты - исключительно как неструктурированные потоки байтов (blob-объекты).

2.3 Разработка алгоритмов синхронизации версий в распределённой среде.

На основе разработанных в разделах 2.1 и 2.2 моделей метаданных (коммит, снимок синхронизации, состояние файла, конфликт) в данном подразделе формализуется алгоритм синхронизации версий между локальным узлом (клиентом) и удалённым узлом (сервером). Алгоритм описывается на концептуальном уровне, без привязки к конкретной реализации, и охватывает все этапы: от инициализации синхронизации до разрешения конфликтов и обновления состояния.

Алгоритм синхронизации строится по принципу "клиент это активная сторона, сервер в свою очередь пассивное хранилище". Клиент инициирует синхронизацию, получает состояние сервера, выполняет локальное объединение (merge), разрешает конфликты (с участием пользователя) и отправляет финальное состояние на сервер. Сервер не участвует в логике merge и разрешения конфликтов — он лишь хранит актуальное состояние файловой системы и отдаёт его по запросу.

Такой подход соответствует архитектуре распределённых систем, описанной Клеррманн М., где каждый узел может автономно работать с локальной копией данных, а синхронизация происходит эпизодически [9]. Tanenbaum A. S. и Van Steen M. подчёркивают, что в распределённых системах с эпизодической связностью активная роль клиента повышает отказоустойчивость и снижает нагрузку на центральные узлы [10].

Алгоритм синхронизации состоит из пяти основных этапов:

1. Инициализация синхронизации и получение состояния сервера.
2. Локальное сравнение и классификация изменений.
3. Автоматическое применение изменений для файлов без конфликтов.
4. Обнаружение и разрешение конфликтов.
5. Формирование финального состояния и отправка на сервер.

Этап 1: Инициализация синхронизации и получение состояния сервера.

Клиент инициирует синхронизацию (по запросу пользователя или по расписанию) и выполняет следующие действия:

Шаг 1.1: Запрос текущего состояния сервера. Клиент отправляет на сервер запрос с указанием своего текущего состояния (идентификатор последнего известного коммита сервера или временную метку). Сервер возвращает:

- Текущий коммит сервера (мастер-состояние) — словарь отображений "путь+имя файла → хеш blob";
- Идентификатор коммита и временную метку;
- Список хешей blob-объектов, которые присутствуют на сервере, но могут отсутствовать локально у клиента.

Шаг 1.2: Загрузка недостающих blob-объектов. Клиент сравнивает полученный список хешей со своим локальным контент-адресуемым хранилищем и определяет, какие blob-объекты отсутствуют локально. Для каждого отсутствующего blob клиент отправляет запрос на сервер и получает содержимое файла. Файл сохраняется в локальное хранилище под именем, равным его хешу.

После завершения этого этапа клиент имеет:

- Полную копию текущего состояния сервера (все blob-объекты загружены локально);
- Свой собственный текущий коммит (локальное состояние файловой системы);
- Снимок последней синхронизации (`last_sync_snapshot`), зафиксированный на момент предыдущей успешной синхронизации.

Этап 2: Локальное сравнение и классификация изменений

Клиент выполняет пофайловое сравнение трёх состояний: снимка синхронизации (`base`), локального состояния (`local`) и состояния сервера (`server`). Для каждого файла, присутствующего хотя бы в одном из трёх состояний, клиент извлекает три хеша:

- `base_hash` - хеш файла в снимке синхронизации (или `null`, если файла не было при последней синхронизации);
- `local_hash` - хеш файла в текущем локальном состоянии (или `null`, если файл отсутствует локально);

- `server_hash` - хеш файла в текущем состоянии сервера (или `null`, если файл отсутствует на сервере).

На основе этих трёх хешей клиент классифицирует состояние файла по правилам, приведённым в таблице 2.3 (раздел 2.1). Классификация определяет, какое действие необходимо выполнить для каждого файла:

1. Синхронизирован (`synced`). Условие: `local_hash == server_hash`. Действие: файл не требует изменений, переход к следующему файлу.
2. Изменён только на сервере (`server_modified`). Условие: `local_hash == base_hash`, `server_hash != base_hash`. Интерпретация: клиент не изменял файл с момента последней синхронизации, но файл был изменён на сервере (другим пользователем или процессом). Действие: принять серверную версию как актуальную. Обновить локальное состояние: записать `server_hash` в локальный коммит.
3. Изменён только локально (`local_modified`). Условие: `local_hash != base_hash`, `server_hash == base_hash`. Интерпретация: клиент изменил файл локально, но на сервере файл не менялся с момента последней синхронизации. Действие: отправить локальную версию на сервер. Записать `local_hash` в серверный коммит (на этапе 5).
4. Конфликт (`conflict`). Условие: `local_hash != base_hash`, `server_hash != base_hash`, `local_hash != server_hash`. Интерпретация: файл был изменён и локально, и на сервере независимо друг от друга. Автоматическое слияние невозможно (особенно для бинарных файлов). Действие: перейти к этапу 4 - разрешение конфликта.
5. Новый файл только локально (`new_local`). Условие: `base_hash == null`, `local_hash != null`, `server_hash == null`. Интерпретация: файл был создан локально после последней синхронизации и отсутствует на сервере. Действие: отправить новый файл на сервер. Записать `local_hash` в серверный коммит (на этапе 5).
6. Новый файл только на сервере (`new_server`). Условие: `base_hash == null`, `local_hash == null`, `server_hash != null`. Интерпретация: файл был создан на сервере после последней синхронизации и отсутствует локально. Действие: принять новый файл локально. Записать `server_hash` в локальный коммит.
7. Файл создан независимо на обоих узлах (`independent_creation`). Условие: `base_hash == null`, `local_hash != null`, `server_hash != null`. Интерпретация: файл с одинаковым путём+именем был создан независимо на клиенте и на сервере (общего предка нет). Это частный случай конфликта. Действие: перейти к этапу 4 — разрешение конфликта.
8. Файл удалён локально, но изменён на сервере. Условие: `local_hash == null`, `base_hash != null`, `server_hash != base_hash`. Интерпретация: клиент удалил файл локально, но на сервере файл был изменён. Действие: конфликт удаления и изменения. Перейти к этапу

- 4 - разрешение конфликта (пользователь должен выбрать: восстановить файл с серверной версией или подтвердить удаление).
9. Файл удалён на сервере, но изменён локально. Условие: `server_hash == null`, `base_hash != null`, `local_hash != base_hash`. Интерпретация: файл был удалён на сервере, но клиент изменил его локально. Действие: конфликт удаления и изменения. Перейти к этапу 4 — разрешение конфликта.
10. Файл удалён на обоих узлах. Условие: `local_hash == null`, `server_hash == null`. Действие: файл отсутствует на обоих узлах, никаких действий не требуется.

Этап 3: Автоматическое применение изменений для файлов без конфликтов

Для файлов со статусами `server_modified`, `local_modified`, `new_local`, `new_server` клиент выполняет автоматические действия:

- Для `server_modified`:
 - Клиент уже загрузил blob с хешем `server_hash` на этапе 1.2;
 - Клиент обновляет свой локальный коммит: записывает `server_hash` для данного пути+имени;
 - Физический файл в локальной файловой системе (если он был доступен пользователю) заменяется на версию с хешем `server_hash`.
- Для `local_modified`:
 - Клиент сохраняет локальные изменения (они уже находятся в локальном коммите)
 - На этапе 5 клиент отправит `local_hash` на сервер.
- Для `new_local`:
 - Клиент сохраняет новый файл в локальном коммите;
 - На этапе 5 клиент отправит `local_hash` и содержимое blob на сервер.
- Для `new_server`:
 - Клиент уже загрузил blob с хешем `server_hash` на этапе 1.2;
 - Клиент добавляет файл в локальный коммит с хешем `server_hash`;
 - Физический файл становится доступным пользователю локально.

После применения всех автоматических изменений локальный коммит клиента обновляется, но ещё не является финальным — остаются конфликтные файлы.

Этап 4: Обнаружение и разрешение конфликтов

Для файлов со статусами `conflict`, `independent_creation`, а также для конфликтов удаления и изменения (статусы 8, 9) клиент переходит к механизму ручного разрешения конфликтов.

Шаг 4.1: Извлечение трёх версий файла. Для каждого конфликтного файла клиент извлекает из контент-адресуемого хранилища три blob-объекта:

- `base_blob` (если `base_hash != null`) — базовая версия, от которой отталкивались оба узла;
- `local_blob` (если `local_hash != null`) — локальная версия пользователя;
- `server_blob` (если `server_hash != null`) — серверная версия.

Шаг 4.2: Создание временных файлов для пользователя. Клиент сохраняет три blob-объекта во временную директорию с именами, отражающими их происхождение:

- `filename_base.ext` — базовая версия (или отсутствует, если `base_hash == null`);
- `filename_local.ext` — локальная версия;
- `filename_server.ext` — серверная версия.

Например, для файла `/courses/math/report.docx` создаются:

- `report_base.docx`
- `report_local.docx`
- `report_server.docx`

Шаг 4.3: Предоставление интерфейса для разрешения конфликта. Клиент отображает пользователю список конфликтных файлов и для каждого файла предоставляет три версии. Пользователь может:

- Открыть каждый файл в соответствующем редакторе (Word, Excel, текстовый редактор и т.д.);
- Сравнить версии вручную;
- Выбрать одну из версий как итоговую;
- Или объединить изменения из разных версий вручную (например, скопировать данные из одного Excel-файла в другой).

Шаг 4.4: Сохранение разрешённой версии. После того как пользователь определил итоговую версию, он сохраняет её как новый файл (например, `report_resolved.docx`). Клиент вычисляет хеш этого файла, создаёт новый blob и сохраняет его в контент-адресуемое хранилище. Клиент обновляет локальный коммит: записывает хеш разрешённой версии для данного пути+имени.

Шаг 4.5: Фиксация разрешения конфликта. Клиент создаёт объект конфликта (модель из раздела 2.1), содержащий:

- Путь+имя файла;
- Хеши трёх версий (`base`, `local`, `server`);
- Хеш разрешённой версии;
- Идентификатор пользователя, разрешившего конфликт;
- Временную метку разрешения.

Этот объект сохраняется в локальную историю для аудита.

Шаг 4.6: Повторение для всех конфликтных файлов. Клиент повторяет шаги 4.1–4.5 для каждого файла из списка конфликтов. После разрешения всех конфликтов локальный коммит клиента становится полностью согласованным и готовым к отправке на сервер.

Этап 5: Формирование финального состояния и отправка на сервер

После завершения этапов 3 и 4 клиент формирует финальное состояние и отправляет его на сервер.

Шаг 5.1: Формирование нового коммита. Клиент создаёт новый коммит, содержащий:

- Словарь файлов (путь+имя → хеш blob) — это обновлённое локальное состояние после применения автоматических изменений и разрешения конфликтов;
- Временную метку;
- Идентификатор автора;
- Комментарий (например, "Sync with server at 2026-03-20 15:00:00").

Шаг 5.2: Определение новых blob-объектов для отправки. Клиент сравнивает хеши из нового коммита с хешами, которые уже есть на сервере (полученными на этапе 1). Клиент формирует список хешей blob-объектов, которые отсутствуют на сервере и должны быть загружены.

Шаг 5.3: Отправка новых blob на сервер. Для каждого нового blob клиент отправляет запрос на сервер с содержимым файла. Сервер сохраняет blob в своё контент-адресуемое хранилище.

Шаг 5.4: Отправка нового коммита на сервер. Клиент отправляет на сервер новый коммит (словарь файлов, метаданные). Сервер проверяет, что мастер-коммит не изменился с момента последнего получения состояния (если изменился — возвращается ошибка, и клиент повторяет синхронизацию с начала). Если проверка пройдена, сервер обновляет мастер-коммит на новый коммит, полученный от клиента, и возвращает подтверждение.

Шаг 5.5: Обновление снимка синхронизации. После получения подтверждения от сервера клиент обновляет свой снимок синхронизации (`last_sync_snapshot`): записывает текущее состояние сервера (которое теперь совпадает с локальным состоянием клиента). Это необходимо для корректной работы алгоритма при следующей синхронизации.

Разработанный алгоритм синхронизации использует несколько архитектурных паттернов, описанных Fowler М. и Gamma Е. и др.:

- Repository (Репозиторий) — клиент взаимодействует с контент-адресуемым хранилищем через абстрактный интерфейс, что позволяет заменить локальное хранилище на распределённое без изменения бизнес-логики [12, 20].

- Unit of Work (Единица работы) — синхронизация рассматривается как единая транзакция: либо все изменения применяются успешно, либо состояние остаётся неизменным [12].
- Optimistic Offline Lock (Оптимистичная блокировка) — клиент предполагает, что конфликты возникают редко, и проверяет согласованность только в момент синхронизации, а не в процессе локальной работы [12].

Таблица 2.3 – состояние файлов на момент синхронизации.

Условие	Статус
`local_hash == server_hash`	synced
`local_hash == base_hash`, `server_hash != base_hash`, `server_hash != null`	server_modified
`local_hash == base_hash`, `server_hash == null`, `base_hash != null`	server_deleted
`local_hash != base_hash`, `server_hash == base_hash`	local_modified
`local_hash != base_hash`, `server_hash != base_hash`, `local_hash != server_hash`	conflict
`base_hash == null`, `local_hash != null`, `server_hash == null`	new_local
`base_hash == null`, `local_hash == null`, `server_hash != null`	new_server
`base_hash == null`, `local_hash != null`, `server_hash != null`	independent_creation
`local_hash == null`, `base_hash != null`, `server_hash != base_hash`	local_deleted_server_modified
`server_hash == null`, `base_hash != null`, `local_hash != base_hash`	server_deleted_local_modified
`local_hash == null`, `server_hash == null`	deleted_both

Таблица 2.3.2 – состояния файлов на момент синхронизации, и описание ситуации.

Статус	Действие
synced	Файл не требует изменений
server_modified	Принять серверную версию
server_deleted	Принять удаление: удалить файл локально
local_modified	Отправить локальную версию на сервер
conflict	Разрешить конфликт вручную
new_local	Отправить новый файл на сервер
new_server	Принять новый файл локально
independent_creation	Разрешить конфликт независимого создания
local_deleted_server_modified	Конфликт: локально удалён, на сервере

	изменён
server_deleted_local_modified	Конфликт: на сервере удалён, локально
deleted_both	изменён Файл отсутствует на обоих узлах

Визуализацию процесса разрешения конфликтов можно посмотреть в Приложении 1.

3 Проектирование интерфейсных решений и архитектуры приложения

В третьем разделе описывается проектирование архитектуры децентрализованной системы хранения неструктурированных данных, схемы базы данных для описания версий на разных узлах и программных интерфейсов взаимодействия узлов синхронизации. Проектирование опирается на модели, разработанные во втором разделе: контент-адресуемое хранилище blob-объектов, коммиты как снимки состояния файловой системы, снимки синхронизации и механизмы разрешения конфликтов.

3.1 Проектирование архитектуры децентрализованной системы хранения неструктурированных данных.

Архитектура системы синхронизации файлов строится по принципу контейнеризированного backend-сервера с браузерным клиентом. Такое решение обеспечивает изоляцию приложения от файловой системы хост-машины, воспроизводимость окружения и упрощает развёртывание. Backend реализован на платформе .NET с использованием фреймворка ASP.NET Core, что соответствует принципам асинхронного взаимодействия и REST-архитектуры, описанным в разделах 1.1 и 1.2.

Система состоит из четырёх основных компонентов:

1. Браузерный клиент (Presentation Layer). Веб-интерфейс, отображаемый в браузере (Chromium или аналогичном). Клиент предоставляет пользователю функции загрузки файлов (кнопка «Add file»), просмотра состояния синхронизации, разрешения конфликтов и управления версиями. Взаимодействие с backend осуществляется через HTTP/REST API.
2. Backend-сервер (Application Layer). ASP.NET Core-приложение, работающее в Docker-контейнере. Отвечает за обработку HTTP-запросов, выполнение бизнес-логики синхронизации, управление контент-адресуемым хранилищем blob-объектов и взаимодействие с базой данных метаданных.
3. База данных метаданных (Data Layer). PostgreSQL 14.3, работающая в отдельном Docker-контейнере. Хранит структурированные метаданные: коммиты (снимки состояния файловой системы), снимки синхронизации (last_sync_snapshot), объекты конфликтов, информацию о пользователях и узлах.
4. Контент-адресуемое хранилище blob-объектов (Storage Layer). Файловая система внутри Docker-контейнера (или подключённый Docker-volume). Хранит физические файлы (blob-объекты), имена которых совпадают с их хеш-суммами (SHA-256). Обеспечивает дедупликацию и целостность данных.

Обоснование выбора технологий.

Платформа .NET и ASP.NET Core. Как отмечается в работе Freeman A., ASP.NET Core предоставляет встроенную поддержку асинхронной обработки запросов, middleware для логирования и аутентификации, а также средства для работы с файлами через IFormFile и Stream [14]. Это критично для системы синхронизации, где операции чтения файлов, вычисления хешей и сетевых запросов выполняются параллельно и не должны блокировать поток обработки запросов.

PostgreSQL. Реляционная база данных выбрана для хранения метаданных по следующим причинам:

- Транзакционность. Операции с коммитами и снимками синхронизации требуют атомарности: либо все изменения применяются, либо состояние остаётся неизменным.
- Согласованность. Связи между коммитами, файлами и хешами должны быть строго формализованы.
- Масштабируемость. PostgreSQL поддерживает индексацию, партиционирование и репликацию, что позволяет развивать систему в будущем.

Docker. Контейнеризация обеспечивает:

- Изоляцию. Backend и база данных работают в изолированных окружениях, что исключает конфликты зависимостей.
- Воспроизводимость. Система развёртывается одинаково на любой машине, поддерживающей Docker.
- Защиту данных. Файлы хранятся внутри контейнера или в Docker-volume, что защищает их от случайного удаления пользователем (как обосновано в разделе 1.1).

Слои архитектуры backend-сервера

Backend-сервер построен по принципам слоистой архитектуры, описанным Fowler M. [12]:

1. Presentation Layer (Контроллеры ASP.NET Core). Отвечает за обработку HTTP-запросов и формирование ответов. Контроллеры не содержат бизнес-логики, а лишь делегируют операции сервисному слою. Основные контроллеры:

- FilesController - загрузка и получение файлов;
- SyncController - операции синхронизации (получение состояния сервера, отправка нового коммита);
- ConflictsController - управление конфликтами;
- CommitsController - просмотр истории коммитов.

2. Application Layer (Сервисы). Содержит бизнес-логику системы. Основные сервисы:

- ISyncService - логика синхронизации (сравнение хешей, классификация изменений, обнаружение конфликтов);

- IblobStorageService - управление контент-адресуемым хранилищем (сохранение, загрузка, проверка существования blob-объектов);

- ICommitService - управление коммитами (создание, получение, обновление);

- IConflictResolutionService - разрешение конфликтов (создание объектов конфликтов, сохранение разрешённых версий).

3. Data Access Layer (Репозитории). Абстрагирует работу с базой данных и файловой системой. Основные репозитории:

- ICommitRepository - операции с коммитами в PostgreSQL;

- ISyncSnapshotRepository - операции со снимками синхронизации;

- IConflictRepository - операции с объектами конфликтов;

- IBlobRepository - операции с файловой системой (чтение, запись blob-объектов).

Такое разделение ответственности соответствует паттерну Repository, описанному Fowler M. [12], и позволяет заменить реализацию хранилища (например, перейти с локальной файловой системы на облачное хранилище) без изменения бизнес-логики.

Архитектура системы синхронизации

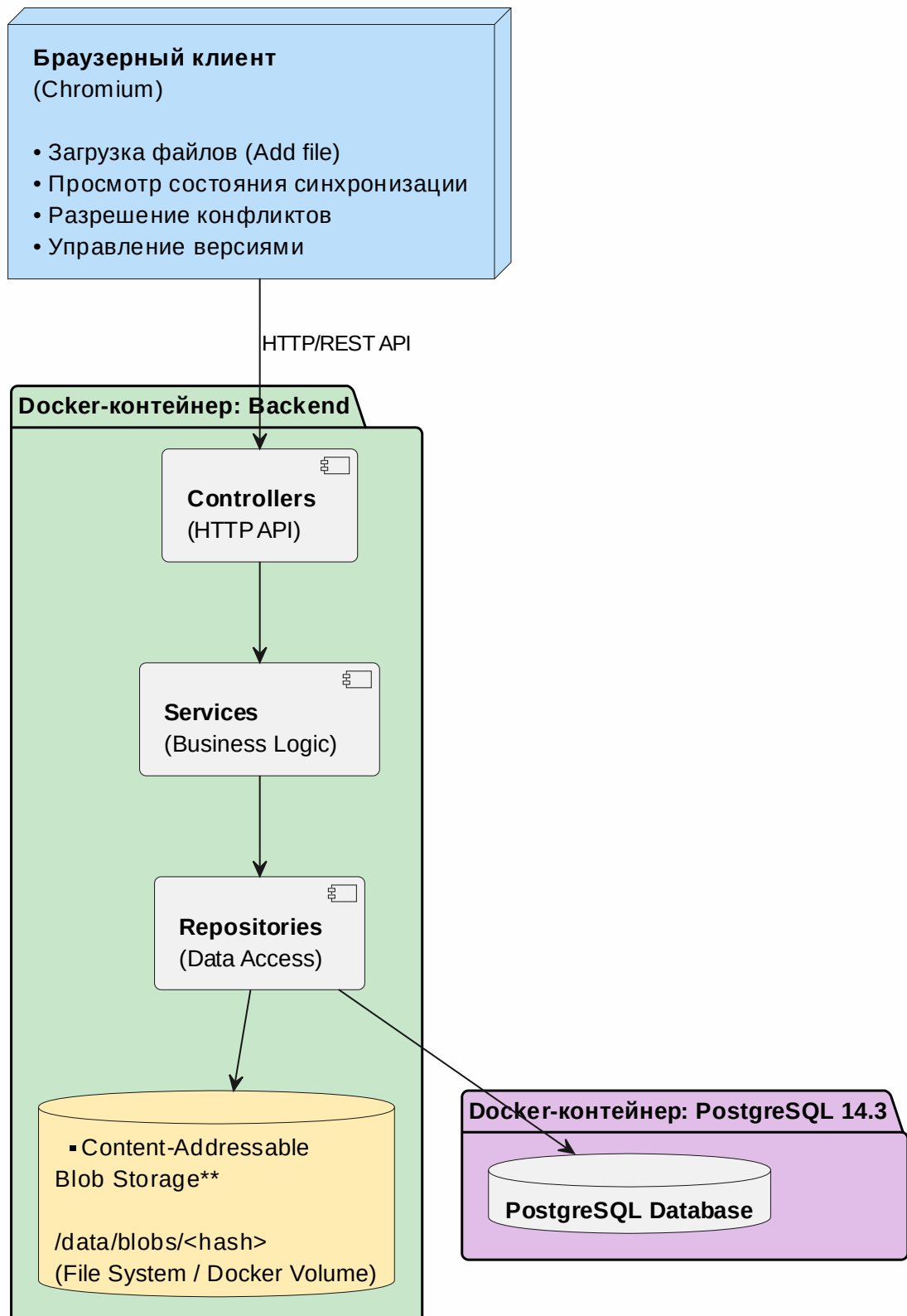


Рисунок 3.1 Архитектура системы синхронизации.

3.2 Проектирование фрагмента схемы базы данных для описания версий на разных узлах.

Для хранения метаданных системы синхронизации файлов выбрана реляционная база данных PostgreSQL 14.3, работающая в изолированном Docker-контейнере. Выбор обусловлен необходимостью обеспечения транзакционности операций с коммитами, согласованности связей между сущностями и поддержки сложных запросов для обнаружения конфликтов и построения истории версий. Как отмечается в работе Klermann M., реляционные СУБД с поддержкой ACID-транзакций остаются оптимальным выбором для систем, требующих строгой согласованности метаданных [9].

ER-модель базы данных

Концептуальная модель базы данных включает шесть основных сущностей, связанных между собой:

1. Users (Пользователи). Хранит информацию о пользователях системы (студенты, преподаватели, администраторы). Каждый пользователь имеет уникальный идентификатор, имя, email и хеш пароля.
2. Blobs (Blob-объекты). Метаданные контент-адресуемого хранилища. Каждый blob идентифицируется по хешу (SHA-256), содержит размер и время создания. Физический файл хранится в файловой системе, а в базе данных — только метаданные.
3. Commits (Коммиты). Снимки состояния файловой системы. Каждый коммит имеет уникальный идентификатор, временную метку, автора, комментарий и ссылку на родительский коммит (для построения истории).
4. CommitFiles (Файлы коммита). Связующая таблица между коммитами и blob-объектами. Реализует словарь "путь+имя файла → хеш blob". Каждая запись содержит идентификатор коммита, путь к файлу и хеш blob-объекта.
5. SyncSnapshots (Снимки синхронизации). Хранит последнее известное состояние сервера для каждого клиента. Содержит идентификатор пользователя, JSON-представление снимка (словарь "путь → хеш") и идентификатор коммита сервера на момент синхронизации.
6. Conflicts (Конфликты). Хранит информацию о разрешённых конфликтах. Содержит путь файла, хеши трёх версий (base, local, server), хеш разрешённой версии, идентификатор пользователя, разрешившего конфликт, и временную метку.

ER-модель базы данных системы синхронизации

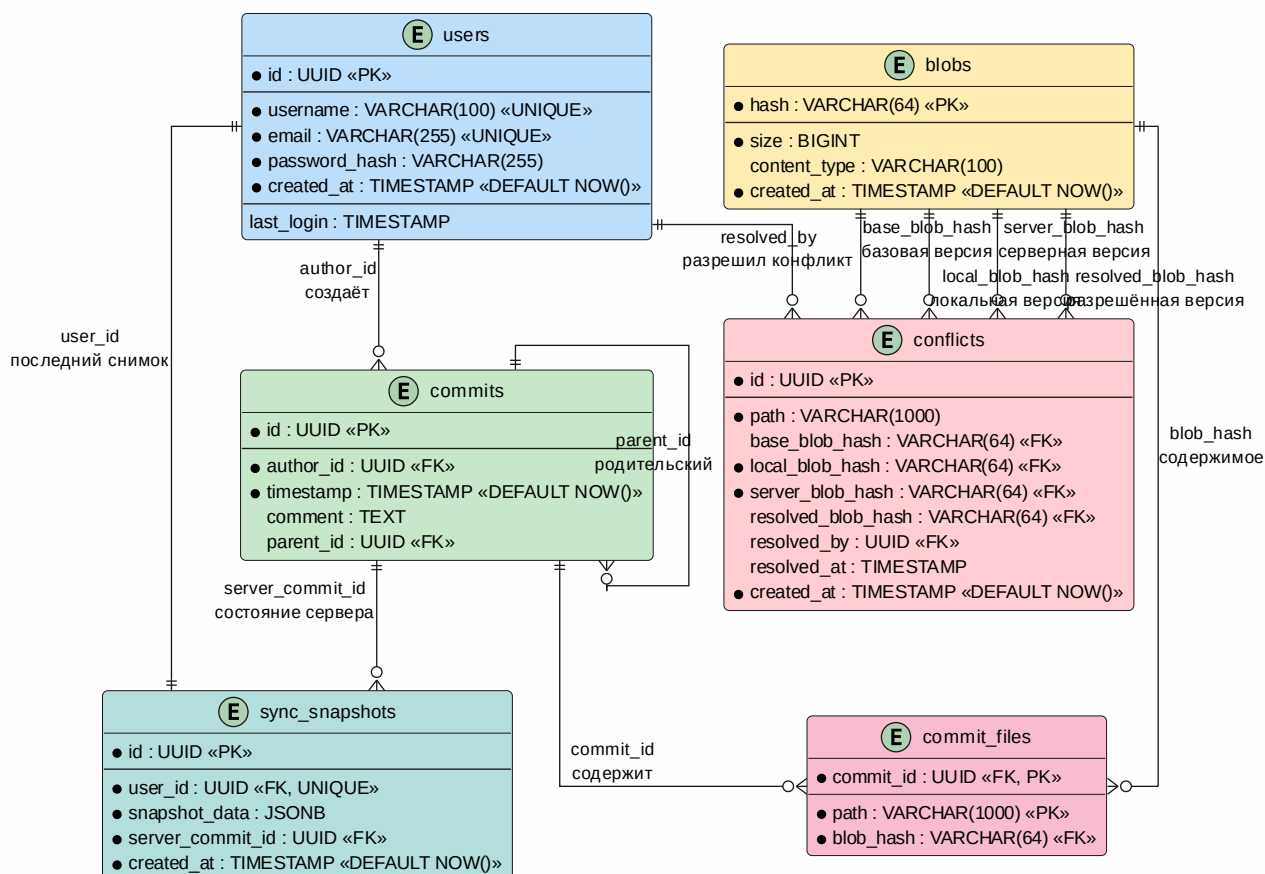


Рисунок 3.2 ER модель базы данных для системы синхронизации.

3.3 Проектирование программных интерфейсов (API) взаимодействия узлов синхронизации

Взаимодействие между узлами системы синхронизации (локальным клиентом и удалённым сервером) осуществляется через REST API, реализованный на основе ASP.NET Core. API обеспечивает выполнение основных операций: загрузку и получение файлов, создание и получение коммитов, синхронизацию состояний, обнаружение и разрешение конфликтов.

API спроектирован в соответствии с принципами REST (Representational State Transfer), описанными Fielding R. T. [13]:

- Stateless - каждый запрос содержит всю необходимую информацию для его обработки;
- Ресурсо-ориентированность - каждый объект (файл, коммит, конфликт) представлен как ресурс с уникальным идентификатором;
- Единый интерфейс - операции выполняются через стандартные HTTP-методы (GET, POST, PUT, DELETE);

- Кэшируемость - ответы могут кэшироваться для повышения производительности.

Все endpoints сгруппированы по функциональным областям и доступны через контроллеры:

- FilesController - операции с файлами (blob-объектами);
- CommitsController - управление коммитами (снимками состояния);
- SyncController - синхронизация между узлами;
- ConflictsController - работа с конфликтами.

Контроллер FilesController предоставляет методы для загрузки, скачивания и проверки существования blob-объектов в контент-адресуемом хранилище.

Метод	Endpoint	Описание
POST	`/api/files/upload`	Загрузка файла в blob-хранилище
GET	`/api/files/{hash}`	Скачивание файла по хешу
GET	`/api/files/{hash}/exists`	Проверка существования blob

1. Загрузка файла (POST /api/files/upload)

Операция принимает файл через multipart/form-data, вычисляет его SHA-256 хеш, проверяет наличие в хранилище (дедупликация) и сохраняет blob под именем, равным хешу. Если файл с таким хешем уже существует, операция возвращает существующий хеш без повторного сохранения.

Алгоритм:

1. Принять файл из запроса;
2. Вычислить SHA-256 хеш содержимого;
3. Проверить наличие blob с таким хешем через IBlobRepository.ExistsAsync();
4. Если blob не существует - сохранить в /data/blobs/<hash>;
5. Создать запись в таблице Blobs (если новая);
6. Вернуть метаданные: хеш, имя файла, размер, MIME-тип.

2. Скачивание файла (GET /api/files/{hash})

Операция извлекает blob из контент-адресуемого хранилища по хешу и возвращает его содержимое как бинарный поток.

Алгоритм:

1. Найти файл по пути /data/blobs/<hash>;
2. Если файл не найден - вернуть HTTP 404;
3. Открыть FileStream с режимом чтения;
4. Вернуть поток с Content-Type: application/octet-stream.

3. Проверка существования (GET /api/files/{hash}/exists)

Операция проверяет наличие blob в хранилище без его загрузки, что используется при

синхронизации для определения необходимых для передачи файлов.

Контроллер `CommitsController` обеспечивает создание, получение и перечисление коммитов - снимков состояния файловой системы.

Метод	Endpoint	Описание
GET	<code>`/api/commits`</code>	Получить все коммиты
GET	<code>`/api/commits/latest`</code>	Получить последний коммит
GET	<code>`/api/commits/{id}`</code>	Получить коммит по ID
GET	<code>`/api/commits/{id}/files`</code>	Получить файлы коммита
POST	<code>`/api/commits`</code>	Создать новый коммит

Контроллер `SyncController` реализует ключевую логику синхронизации между локальным и удалённым узлами.

Метод	Endpoint	Описание
GET	<code>`/api/sync/state`</code>	Получить текущее состояние сервера
POST	<code>`/api/sync/sync`</code>	Выполнить синхронизацию

1. Получение состояния сервера (GET `/api/sync/state`)

Операция возвращает текущий мастер-коммит сервера в виде словаря "путь → хеш", что позволяет клиенту сравнить своё состояние с серверным.

Алгоритм:

1. Получить последний коммит через `ICommitService.GetLatestCommitAsync()`;
2. Если коммитов нет - вернуть пустое состояние;
3. Извлечь файлы через `ICommitService.GetCommitFilesAsync(commitId)`;
4. Преобразовать в `Dictionary<string, string>` (path → blob_hash);
5. Вернуть `ServerStateDto` с `commitId`, `timestamp`, `files`.

2. Выполнение синхронизации (POST `/api/sync/sync`)

Операция принимает состояние клиента, сравнивает его с серверным, обнаруживает конфликты и создаёт новый коммит с разрешёнными изменениями.

Алгоритм:

1. Получить текущее состояние сервера (`serverState`);
2. Получить снимок последней синхронизации для `userId` из `SyncSnapshots (baseState)`;
3. Для каждого файла из объединения ключей `clientState` и `serverState`:
4. Извлечь `base_hash` из `baseState` (или `null`);
5. Извлечь `local_hash` из `clientState` (или `null`);
6. Извлечь `server_hash` из `serverState` (или `null`);
7. Классифицировать статус по правилам из раздела 2.3;
8. При обнаружении конфликта (статус `conflict`) - создать запись в `Conflicts`;

9. Сформировать newState - итоговый словарь "путь -> хеш" после применения правил слияния;
10. Создать новый коммит через ICommitService.CreateCommitAsync();
11. Обновить SyncSnapshot для userId;
12. Вернуть SyncResultDto с результатом.

Контроллер ConflictsController обеспечивает получение списка неразрешённых конфликтов и фиксацию их разрешения.

Метод	Endpoint	Описание
GET	`/api/conflicts/unresolved`	Получить неразрешённые конфликты
POST	`/api/conflicts`	Создать запись о конфликте
POST	`/api/conflicts/{id}/resolve`	Разрешить конфликт

1. Получение неразрешённых конфликтов (GET /api/conflicts/unresolved)

Операция возвращает список конфликтов, у которых resolvedBlobHash IS NULL, что используется клиентом для отображения пользователю файлов, требующих ручного разрешения.

2. Разрешение конфликта (POST /api/conflicts/{id}/resolve)

Операция фиксирует результат ручного разрешения конфликта, сохраняя хеш итоговой версии.

4 Реализация прототипа, тестирование и наполнение данными

В четвёртом разделе описывается программный прототип системы синхронизации файлов, реализованный как контейнеризированное приложение на платформе .NET с браузерным клиентом. Прототип не претендует на завершённую промышленную информационную систему, но демонстрирует основные архитектурные решения и подтверждает возможность практической реализации моделей, разработанных во втором и третьем разделах: контент-адресуемого хранилища, снимков состояния файловой системы, алгоритмов синхронизации и механизмов разрешения конфликтов.

4.1 Разработка программного прототипа системы распределённого хранения и синхронизации.

Реализованный прототип состоит из двух контейнеров, развёртываемых через Docker Compose: backend-сервера на ASP.NET Core 10 и базы данных PostgreSQL 14.3. Браузерный клиент представлен как статическое одностраничное приложение (SPA), раздаваемое самим backend через механизм UseStaticFiles.

В качестве технологической основы используются платформа .NET 10, язык C#, фреймворк ASP.NET Core, PostgreSQL и библиотека Swashbuckle.AspNetCore для автоматической генерации документации API в формате OpenAPI 3.0. Такой выбор соответствует принципам асинхронного взаимодействия и слоистой архитектуры, описанным в работах Fowler M. [12] и Freeman A. [14].

Архитектура backend построена по принципам слоистой архитектуры с разделением ответственности:

Presentation Layer - контроллеры ASP.NET Core (FilesController, CommitsController, SyncController, ConflictsController);

- Application Layer - сервисы бизнес-логики (IBlobStorageService, ICommitService, ISyncService, IConflictResolutionService);
- Data Access Layer - репозитории для работы с БД и файловой системой (ICommitRepository, IBlobRepository, ISyncSnapshotRepository, IConflictRepository);
- Storage Layer - контент-адресуемое хранилище blob-объектов в файловой системе контейнера (том blob-storage).

4.2 Механизмы загрузки данных и наполнение тестовыми файлами.

Files API (/api/files/upload, /api/files/{hash}, /api/files/{hash}/exists) — операции с контент-адресуемым хранилищем;

- Commits API (/api/commits, /api/commits/latest, /api/commits/{id}, /api/commits/{id}/files) — управление снимками состояния;
- Sync API (/api/sync/state, /api/sync/sync) — синхронизация между узлами;
- Conflicts API (/api/conflicts/unresolved, /api/conflicts, /api/conflicts/{id}/resolve) — работа с конфликтами.

Для проверки системы был реализован сценарий наполнения прототипа тестовыми данными, соответствующий учебному контексту: учебные документы (Excel-таблицы с расписанием и журналом успеваемости), текстовые файлы лабораторных работ, презентации PowerPoint.

Тестовый сценарий включал следующие шаги:

1. Загрузка файла через браузерный интерфейс (POST /api/files/upload);
2. Получение хеша из ответа API (например, c98c24b677eff44860afea6f493bbaec5bb1c4cbb209c6fc2bbb47f66ff2ad31);
3. Создание коммита с указанием словаря path → hash (POST /api/commits);
4. Получение истории коммитов через GET /api/commits;
5. Скачивание файла по хешу (GET /api/files/{hash}) для проверки целостности.

4.3 Отладка, функциональное тестирование и оценка производительности.

Отладка прототипа выполнялась с использованием двухуровневого подхода к тестированию, опирающегося на автоматически сгенерированную документацию API и пользовательский графический интерфейс.

Тестирование программных интерфейсов через Swagger UI. Для верификации каждого endpoint системы использовался интерактивный веб-интерфейс Swagger UI, автоматически генерируемый библиотекой Swashbuckle.AspNetCore на основе спецификации OpenAPI 3.0. Данный инструмент предоставляет полный перечень endpoints, сгруппированных по функциональным областям (Files, Commits, Sync, Conflicts), с возможностью выполнения HTTP-запросов непосредственно из браузера, наблюдения форматов запросов и ответов, а также проверки кодов состояний. Все сведения о программных интерфейсах — схемах входных параметров, моделях данных, возможных кодах ответов — доступны в Swagger UI без необходимости изучения исходного кода, что соответствует принципам самодокументируемых API, описанным в спецификации OpenAPI [21].

Тестирование пользовательских сценариев через графический интерфейс. Дополнительно функциональное тестирование проводилось через браузерный графический интерфейс (GUI), реализованный как одностраничное HTML/JavaScript-приложение в каталоге wwwroot/index.html и взаимодействующее с backend через REST API. GUI покрывает

следующие пользовательские сценарии: загрузку файлов через кнопку «Add file», создание коммита через кнопку «Make commit», просмотр истории коммитов и отображение метаданных файлов (имя, размер, SHA-256 хеш).

Сочетание двух уровней тестирования позволяет отделить проверку корректности программных интерфейсов от проверки пользовательских сценариев, что соответствует принципу разделения ответственности [21].

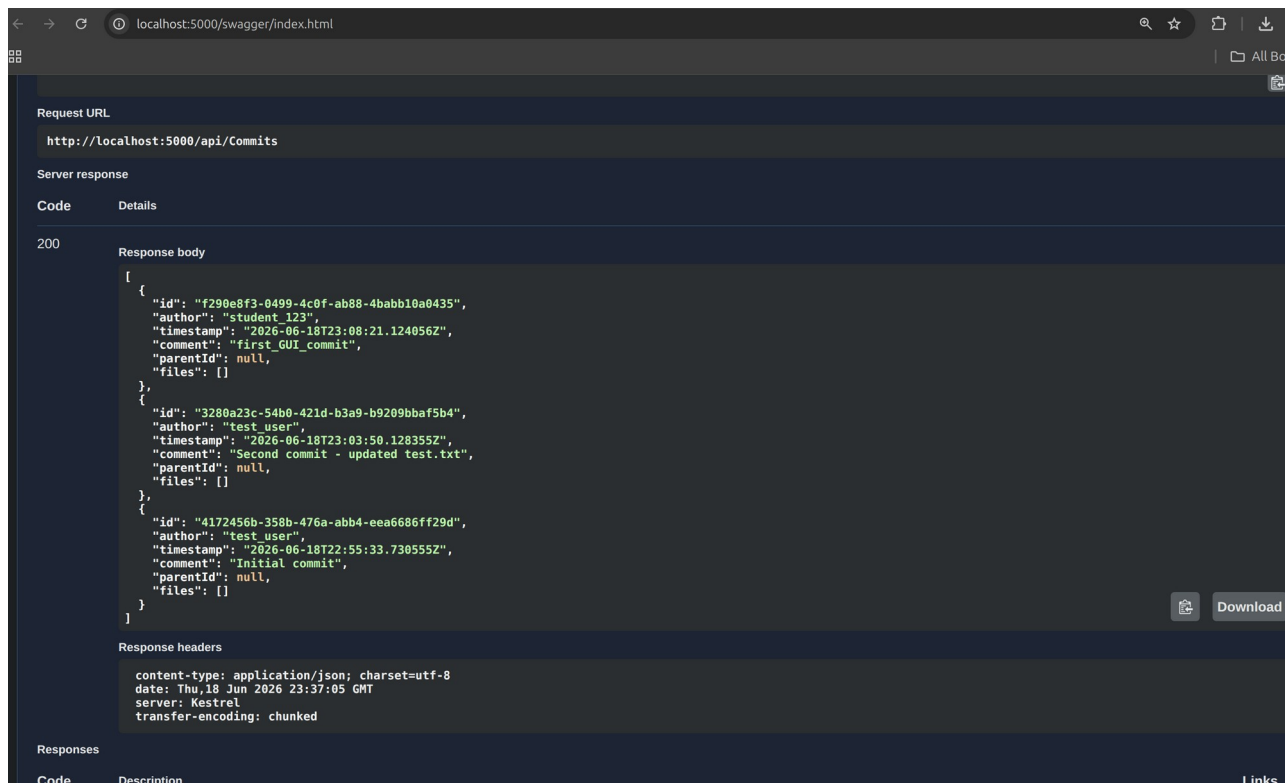


Рисунок 4.3.1 – Иллюстрация тестирования программы в Swagger

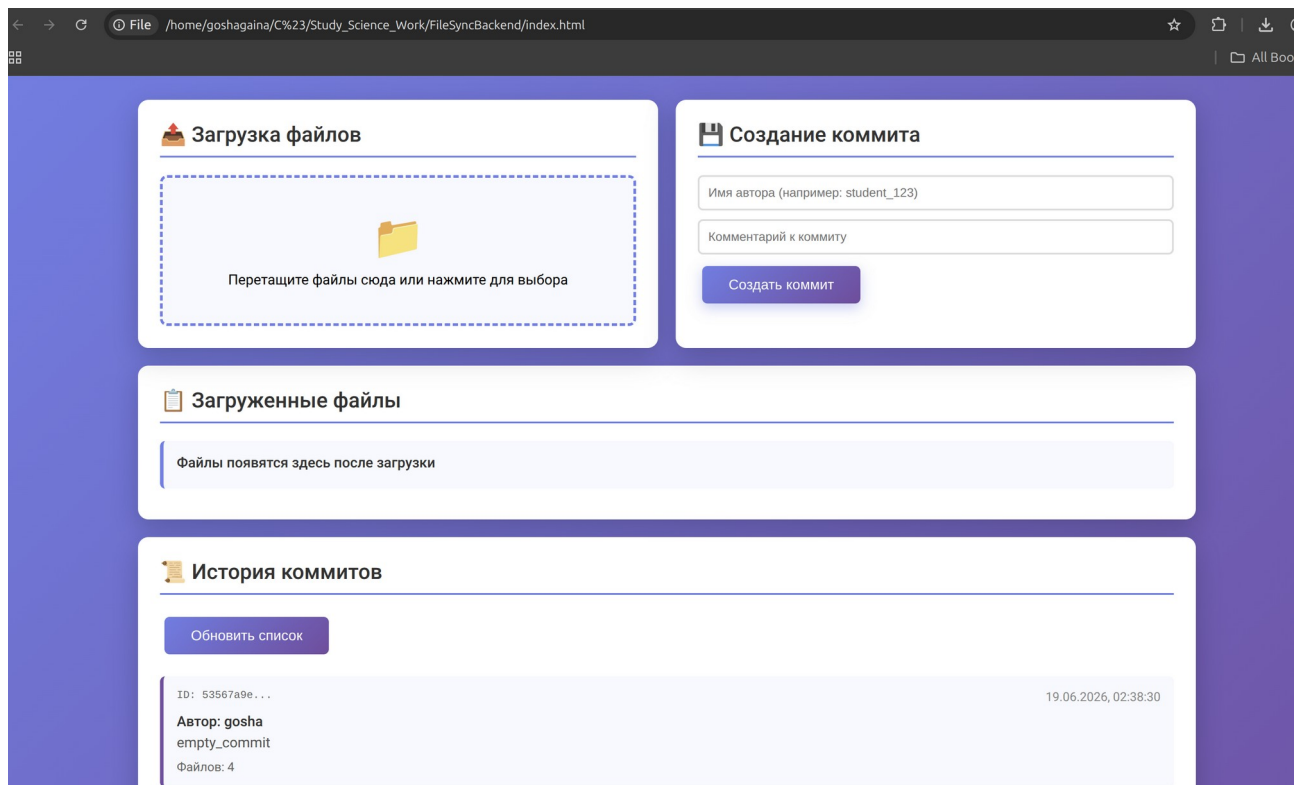


Рисунок 4.3.2 Иллюстрация тестирования программы в браузерном GUI.

Заключение

В ходе учебно-исследовательской работы была решена задача разработки прототипа системы синхронизации файлов для системы поддержки учебного процесса. Основной упор при разработке был сделан на создание надёжной распределённой архитектуры, обеспечивающей защиту данных, их целостность и прозрачное управление конфликтами на различных узлах.

В первом разделе был проведён анализ возможностей платформы .NET. Для обеспечения защиты данных и целостности версионирования было принято решение отказаться от прямого доступа к файловой системе хоста в пользу изолированного хранилища (Docker-volume). Подтверждена эффективность асинхронного взаимодействия через HttpClient и паттерн async/await. Проведённый сравнительный анализ Git и SVN позволил разработать гибридную модель, включающую контент-адресуемость Git и механизм трёхстороннего разрешения конфликтов SVN, что является оптимальным для работы с бинарными учебными файлами.

Во втором разделе была предложена и формализована модель метаданных для хранения и версионирования данных. Разработанная модель включает неизменяемые контент-адресуемые blob-объекты (SHA-256), снимки состояния файловой системы (коммиты) и классификаторы статуса файлов (синхронизирован, изменён локально, изменён на сервере, конфликт и др.). Сформулированы алгоритмы синхронизации, основанные на сравнении трёх хеш-сумм (базовой, локальной и серверной), а также механизмы автоматического слияния и ручного разрешения конфликтов.

В третьем разделе была спроектирована архитектура системы хранения неструктурированных данных, построенная по принципу контейнеризированного бэкенда (ASP.NET Core, Docker) с браузерным клиентом. Разработана ER-модель базы данных PostgreSQL, включающая сущности пользователей, коммитов, файлов и конфликтов. Спроектирован REST API, обеспечивающий загрузку файлов, управление версиями (коммитами), синхронизацию состояний и разрешение конфликтов.

В четвёртом разделе описан практическая реализация программного прототипа. Прототип развёрнут в Docker-контейнерах, включает функциональные контроллеры и сервисы для работы с файлами, синхронизацией и конфликтами. Проведена отладка и функциональное тестирование с использованием двухуровневого подхода: через Swagger UI для проверки корректности API и через браузерный графический интерфейс для отработки пользовательских сценариев. Работоспособность системы подтверждена на тестовых данных, имитирующих учебный контент (лабораторные работы, расписания, методические материалы).

Таким образом, цель работы достигнута: разработан прототип системы синхронизации файлов, который обеспечивает распределённое хранение данных, контроль версий и разрешение конфликтов. Дальнейшее развитие работы связано с внедрением gRPC для повышения производительности, интеграцией с облачными объектными хранилищами и разработкой полноценного клиентского интерфейса, обеспечивающего удобную работу пользователей с распределёнными учебными материалами.

Список литературы

1. *Методические указания для студентов к оформлению пояснительной записки к УИР/НИР.* — М.: НИЯУ МИФИ, кафедра кибернетики.
2. ГОСТ 19.101-77. Единая система программной документации. Виды программ и программных документов.
3. ГОСТ 19.102-77. Единая система программной документации. Стадии разработки.
4. ГОСТ 19.701-90. Единая система программной документации. Схемы алгоритмов, программ, данных и систем.
5. ГОСТ 19781-90. Обеспечение систем обработки информации программное. Термины и определения.
6. ГОСТ 15971-90. Системы обработки информации. Термины и определения.
7. Troelsen A., Japikse P. *Pro C# 10 with .NET 6: Foundational Principles and Practices in Programming.* — Apress, 2022. — 1400 p.
8. Richter J. *CLR via C#.* — 4th ed. — Microsoft Press, 2012. — 896 p.
9. Kleppmann M. *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems.* — Sebastopol: O'Reilly Media, 2017. — 616 p.
10. Tanenbaum A. S., Van Steen M. *Distributed Systems: Principles and Paradigms.* — 3rd ed. — Pearson, 2017. — 518 p.
11. Albahari J., Albahari B. *C# 12 in a Nutshell: The Definitive Reference.* — O'Reilly Media, 2023. — 1000 p.
12. Fowler M. *Patterns of Enterprise Application Architecture.* — Boston: Addison-Wesley, 2002. — 560 p.
13. Fielding R. T. *Architectural Styles and the Design of Network-based Software Architectures.* — University of California, Irvine, 2000. — 260 p.
14. Freeman A. *Pro ASP.NET Core 6: The Definitive Guide to Building Web Applications.* — Apress, 2022. — 1200 p.
15. Калмыков А. Ю., Синкевич Н. В., Денисов В. В. *Архитектура клиент-сервер // Прошлое, настоящее и будущее российской цивилизации.* — 2016. — С. 164.
16. Newman S. *Building Microservices: Designing Fine-Grained Systems.* — 2nd ed. — O'Reilly Media, 2021. — 366 p.
17. Richardson C. *Microservices Patterns: With examples in Java.* — Manning Publications, 2018. — 552 p.
18. Sommerville I. *Software Engineering.* — 10th ed. — Harlow: Pearson, 2016. — 944 p.
19. Chacon S., Straub B. *Pro Git.* — 2nd ed. — Apress, 2014. — 458 p.

20. Gamma E., Helm R., Johnson R., Vlissides J. Design Patterns: Elements of Reusable Object-Oriented Software. — Boston: Addison-Wesley, 1994. — 395 p.
21. OpenAPI Specification (версия 3.0.0).

Приложение А. Демонстрация процесса разрешения конфликтов.

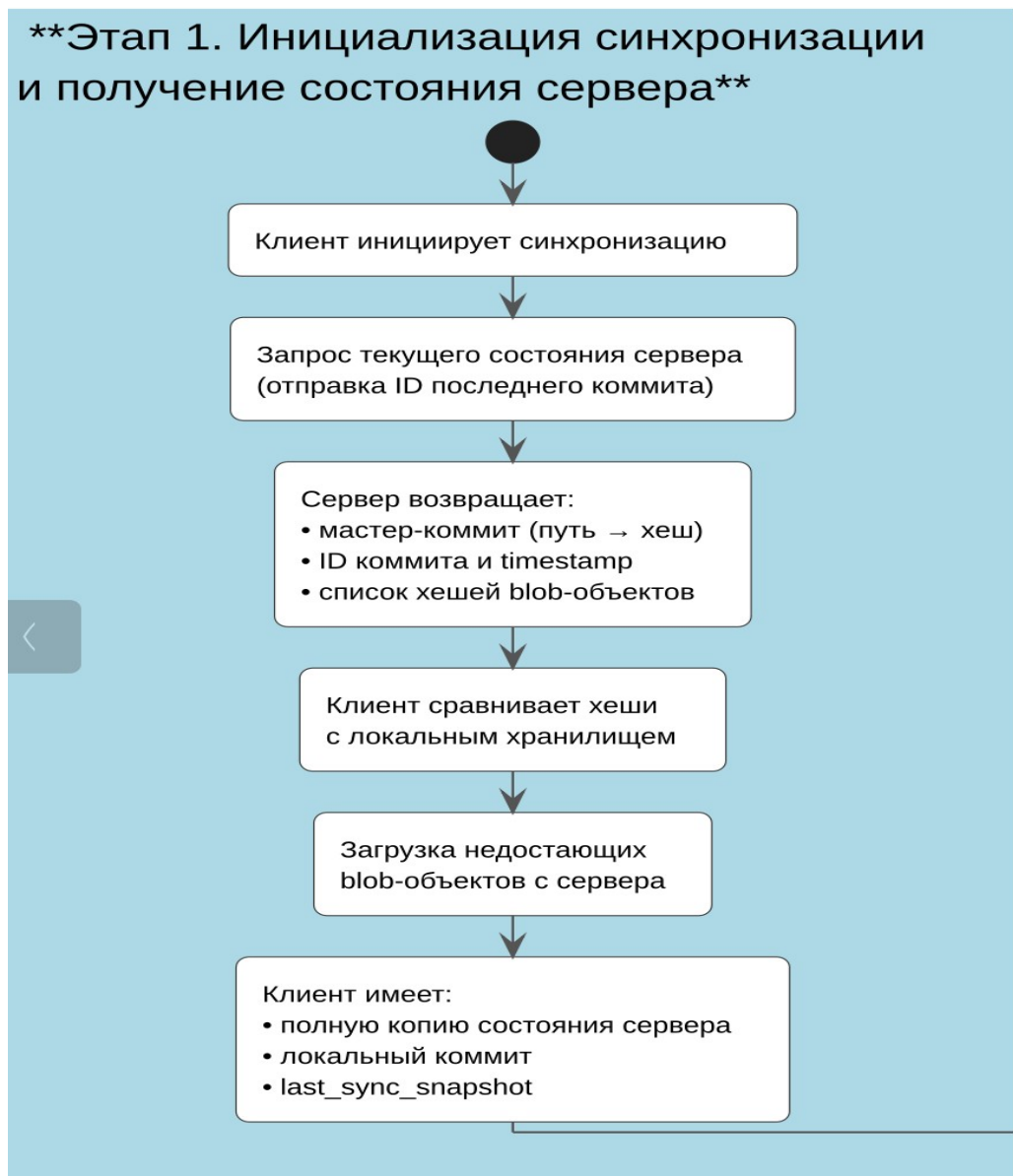


Рисунок А.1 – Инициализация синхронизации клиента и сервера.

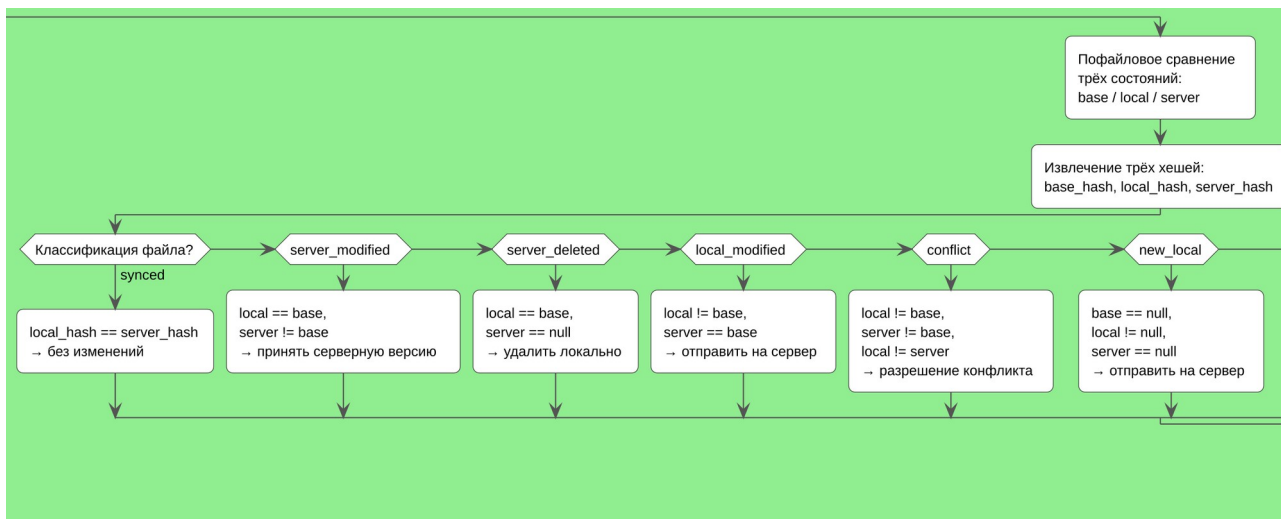


Рисунок А.2 - Локальное сравнение хешей и классификация изменений.

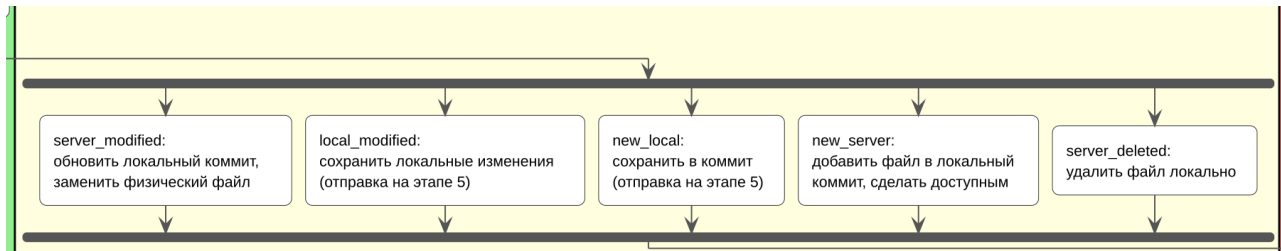


Рисунок А.3 – Автоматическое применение изменений к локальному хранилищу.

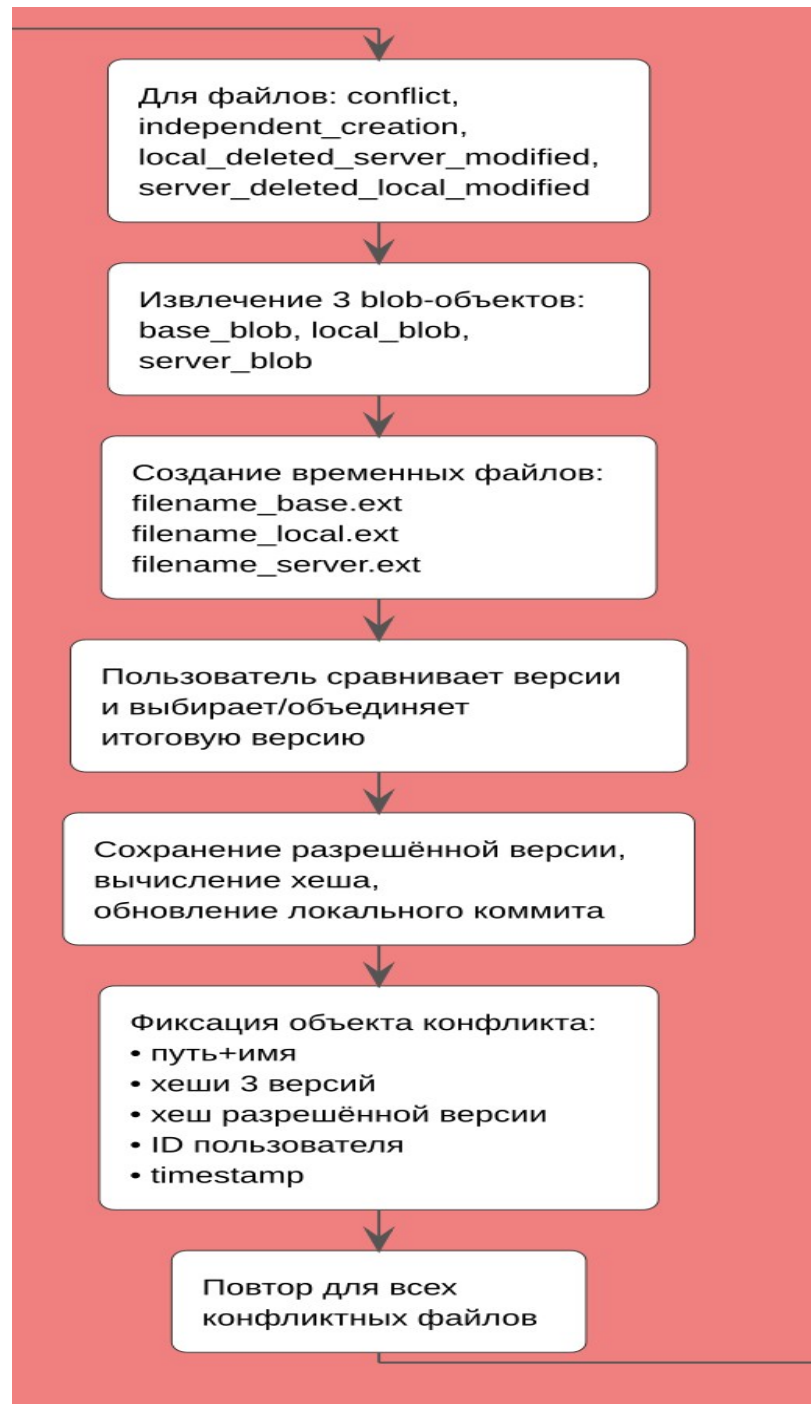


Рисунок А.4 – Обнаружение и разрешение конфликтов.

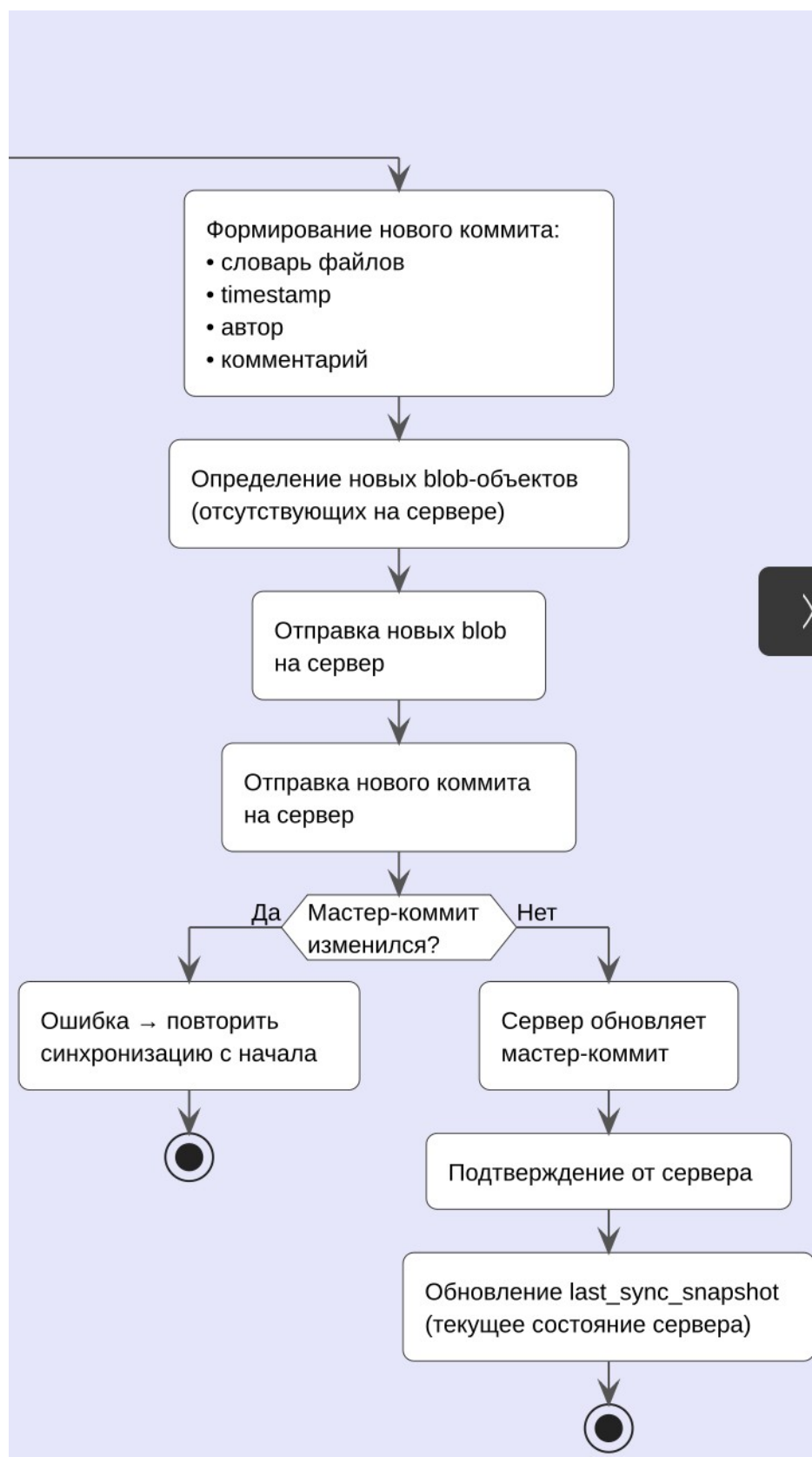


Рисунок А.5 Формирование финального состояния и отправка на сервер.